

EnergyGrid

Smart Energy & Utilities Management
System

Complete Interview Preparation Guide

Project	EnergyGrid — Smart Energy & Utilities Management System
Tech Stack	Java Spring Boot 3 · MySQL 8.4 · React v18 · Bootstrap 5
Batch	Batch 11 · PoD 3 · INTDE26JFSR011 · 5 Associates
Interim Eval	26–29 Jun 2026
Final Eval	12–18 Aug 2026
Document	Project Overview · 5 Module-wise Details · Backend Q&A · Frontend Q&A · Spring Boot Deep Dive

Emp ID	Name	Backend Module	Frontend Module
2507351	Vinnakota Bala Adithya Vardan	IAM + Energy Analytics & Reporting	Login + Admin Console + Analytics Dashboard
2507325	Palleti Nagesh Varma	Billing & Tariff Management	Billing Dashboard + Dispute Management UI
2507384	Ponnamanda Raghu Vamsi	Consumer Account + Connection + Meter Data	Consumer Portal + Meter Reading UI
2507286	Vakada Srinu	Payments & Collections + Notifications	Payment Portal + Notifications UI
2507337	Sai Sowmya Kotturi	Outage & Grid Event Management	Grid Ops Dashboard + Outage Tracker

SECTION 1 — PROJECT OVERVIEW & ARCHITECTURE

Project Description

EnergyGrid is a full-stack Smart Energy and Utilities Management System built for PoD 3, Batch 11. It manages the complete lifecycle of an electricity utility company: consumer onboarding, meter installation, billing generation, payment collection, outage tracking, field dispatch, and compliance analytics reporting. Backend is a monolithic Spring Boot 3 REST API on port 8095. Frontend is a React v18 SPA with Bootstrap 5, Axios, and React Router DOM.

Complete Technology Stack

- Backend: Java 17 + Spring Boot 3.2.5 + Spring Data JPA + Spring Security 6 + Spring AOP
- Database: MySQL 8.4 — schema: energygrid_monolithic_db — port 3306
- ORM: Hibernate — ddl-auto=update — MySQL8Dialect — show-sql=true
- Authentication: JWT (jjwt 0.11.5) — stateless Bearer token — 24 hour expiry
- Password Security: BCryptPasswordEncoder from Spring Security
- Frontend: React v18 + Bootstrap 5 + React Router DOM v6 + Axios
- State Management: React Context API (AuthContext) + useState + useEffect hooks
- API Communication: Axios with request interceptor auto-appending Authorization: Bearer token
- Logging: SLF4J + @Slf4j (Lombok) in ALL Service classes
- AOP: LoggingAspect — @Before + @AfterReturning on all service.impl.* methods
- Testing: JUnit 5 + Mockito — @ExtendWith(MockitoExtension.class) + @Mock + @InjectMocks
- Build: Maven (backend) — npm / create-react-app (frontend)

All 15 Database Tables — Complete Reference

Table Name	Primary Key	Foreign Key(s)	Enum Status Values
users	UserID (PK)	—	ACTIVE / INACTIVE
audit_logs	AuditID (PK)	UserID → users	— (Action VARCHAR 50)
energy_reports	ReportID (PK)	—	Scope: ZONE/CATEGORY/PERIOD/GRID
tariffs	TariffID (PK)	—	ACTIVE / EXPIRED Category: RESIDENTIAL/COMMERCIAL/INDUSTRIAL
bills	BillID (PK)	TariffID → tariffs	GENERATED / SENT / PAID / OVERDUE / DISPUTED
billing_disputes	DisputeID (PK)	BillID → bills	OPEN / UNDERREVIEW / RESOLVED / REJECTED
consumer_accounts	AccountID (PK)	—	ACTIVE / SUSPENDED / DISCONNECTED ConnType: RESIDENTIAL/COMMERCIAL/INDUSTRIAL
connection_requests	RequestID (PK)	—	PENDING / SCHEDULED / COMPLETED / REJECTED Type: NEWCONNECTION/UPGRADE/DISCONNECTION/RECONNECTION
meters	MeterID (PK)	AccountID → consumer_accounts	ACTIVE / FAULTY / REPLACED Type: ANALOG/DIGITAL/SMARTAMI
meter_readings	ReadingID (PK)	MeterID → meters	VALIDATED / ESTIMATED / DISPUTED Source: MANUAL/AMI/ESTIMATED
payments	PaymentID (PK)	—	PENDING / RECEIVED / FAILED Method: ONLINE/CASH/CHEQUE/AUTODEBIT

installment_plans	PlanID (PK)	—	ACTIVE / COMPLETED / DEFAULTED
notifications	NotificationID (PK)	—	UNREAD / READ / DISMISSED Category: BILLING/PAYMENT/OUTAGE/METER/COMPLIANCE
outage_events	OutageID (PK)	—	ACTIVE / RESTORED / INVESTIGATING Cause: PLANNED/UNPLANNED/WEATHER/EQUIPMENT
field_dispatches	DispatchID (PK)	OutageID → outage_events	DISPATCHED / ONSITE / RESOLVED

All REST API Endpoints — Quick Reference

Module	Method	Endpoint	Response / Notes
Auth	POST	/api/auth/login	Returns {token, role, userId} or 401
Users	POST	/api/users	Create user → 201
Users	GET	/api/users	List all users
Users	GET	/api/users/{id}	User by ID → 200 or 404
Users	PATCH	/api/users/{id}?status=	Update status → 200
Users	DELETE	/api/users/{id}	Soft deactivate → 204
Tariff	POST	/api/tariffs	Create tariff → 201
Tariff	GET	/api/tariffs?status=&category=	Filter tariffs → list
Tariff	PATCH	/api/tariffs/{id}/status	Update tariff status
Bills	POST	/api/bills	Generate bill (calc amount) → 201
Bills	GET	/api/bills?accountId=	Bills by account
Bills	PATCH	/api/bills/{id}/status	Update bill status
Bills	PATCH	/api/bills/mark-overdue	Batch: GENERATED past DueDate → OVERDUE
Disputes	POST	/api/billing-disputes	Raise dispute → 201, status=OPEN
Disputes	GET	/api/billing-disputes?billId=&status=	Filter disputes
Disputes	PATCH	/api/billing-disputes/{id}/resolve	Resolve dispute
Disputes	PATCH	/api/billing-disputes/{id}/reject	Reject dispute
Consumer	POST	/api/consumer-accounts	Register consumer → 201
Consumer	GET	/api/consumer-accounts?consumerId=	Consumer accounts
Consumer	PATCH	/api/consumer-accounts/{id}/status	Update account status
Connection	POST	/api/connection-requests	Submit request → 201, PENDING
Connection	PATCH	/api/connection-requests/{id}/assign?techId=	Assign tech → SCHEDULED
Connection	PATCH	/api/connection-requests/{id}/status	Update request status
Meter	POST	/api/meters	Install meter → 201 (400 if acctId invalid)
Meter	GET	/api/meters?accountId=	Meters by account
Meter	PATCH	/api/meters/{id}/status	Update meter status
Reading	POST	/api/meter-readings	Record reading → 201
Reading	GET	/api/meter-readings?meterId=	Readings by meter
Reading	GET	/api/meter-readings/meter/{id}/latest	Latest reading for meter
Reading	GET	/api/meter-readings/meter/{id}/consumption?from=&to=	Consumption calc (kWh)

Payment	POST	/api/payments	Record payment → 201 (400 if already PAID)
Payment	GET	/api/payments?accountId=	Payments by account
Installment	POST	/api/installment-plans	Create plan → 201
Installment	PATCH	/api/installment-plans/{id}/complete	Mark COMPLETED
Installment	PATCH	/api/installment-plans/{id}/default	Mark DEFAULTED
Notification	POST	/api/notifications	Send notification → 201, UNREAD
Notification	GET	/api/notifications/user/{id}	All notifications for user
Notification	GET	/api/notifications/user/{id}/unread	Unread only
Notification	PATCH	/api/notifications/{id}/read	Mark single as READ
Notification	PATCH	/api/notifications/user/{id}/read-all	Mark all read → {updated: N}
Notification	GET	/api/notifications/user/{id}/count	Returns {unreadCount: N}
Outage	POST	/api/outage-events	Report outage → 201, ACTIVE
Outage	GET	/api/outage-events?status=&zoneId=	Filter outages
Outage	GET	/api/outage-events/active	All ACTIVE outages
Outage	PATCH	/api/outage-events/{id}/restore	Restore → 200 (400 if already RESTORED)
Outage	GET	/api/outage-events/{id}/sla	SLA: {estimated, actual, slaMet: bool}
Dispatch	POST	/api/field-dispatches	Dispatch tech → 201 (400 if invalid outageId)
Dispatch	PATCH	/api/field-dispatches/{id}/status	DISPATCHED > ONSITE > RESOLVED
Dispatch	PATCH	/api/field-dispatches/{id}/resolve	Resolve dispatch + set resolutionTime
AuditLog	GET	/api/audit-logs?userId=&action=&from=&to=	Pageable filtered audit log
Reports	POST	/api/energy-reports?scope=	Generate report → 201
Reports	GET	/api/energy-reports	All reports
Reports	GET	/api/energy-reports/summary	KPI summary for dashboard

SECTION 2 — MODULE-WISE: DETAILED EXPLANATION & Q&A; (PERSON BY PERSON)

Vinnakota Bala Adithya Vardan

ID: 2507351

Backend: IAM + Energy Analytics & Reporting

Frontend: Login Page + Admin Console +
Analytics Dashboard

HOW TO EXPLAIN YOUR MODULE TO THE INTERVIEWER

Backend — IAM (Identity and Access Management): I built the entire authentication and user management backbone of EnergyGrid. My module covers three database tables — users, audit_logs, and energy_reports. The User entity has 6 roles: CONSUMER, FIELD_TECH, BILLING, GRID_OPS, COMPLIANCE, and ADMIN. I implemented JWT-based stateless login where the client sends email and password, receives a signed token with role claim, and sends that token in the Authorization header on every subsequent request. Passwords are hashed using BCrypt — never stored in plain text. I implemented soft-delete for user deactivation so historical data is preserved. AuditLogService is a shared service called by every other module to record significant actions — it has a single method logAction(userId, action, entityType, entityId). EnergyReportService aggregates data from audit logs and bills to produce JSON-based metrics reports scoped by ZONE, CATEGORY, PERIOD, or GRID.

Frontend — Login + Admin Console + Analytics Dashboard: I built LoginPage.jsx with a Bootstrap form that submits to POST /api/auth/login. After successful login, I store the JWT token, role, and userId in localStorage via AuthContext. The axios instance in api.js has a request interceptor that automatically reads the token from localStorage and appends 'Authorization: Bearer token' to every outgoing request. The Layout.jsx component renders a role-based sidebar — each role sees only their permitted menu items. UserListPage has a Bootstrap table with search and filter by role, and a UserFormModal for adding users. The AnalyticsDashboard shows 4 KPI cards populated from GET /api/energy-reports/summary.

BACKEND INTERVIEW QUESTIONS — IAM + Energy Analytics

Q1. Walk me through your IAM module's database schema.

A: I have three tables. users: UserID (PK, auto-increment), Name (VARCHAR 100), Role (ENUM: CONSUMER/FIELD_TECH/BILLING/GRID_OPS/COMPLIANCE/ADMIN stored as STRING), Email (VARCHAR UNIQUE), Phone (VARCHAR), ZoneID (INT), Status (ENUM: ACTIVE/INACTIVE). audit_logs: AuditID (PK), UserID (FK to users), Action (VARCHAR 50), EntityType (VARCHAR), EntityID (INT), Timestamp (DATETIME DEFAULT NOW()). energy_reports: ReportID (PK), Scope (VARCHAR 50), Metrics (TEXT — stores JSON string), GeneratedDate (DATETIME DEFAULT NOW()). I seeded 5 users (one per role), 3 audit logs, and 2 reports for initial testing and Postman validation.

Q2. Why did you use 6 roles instead of just ADMIN and USER?

A: The domain demands fine-grained access control that mirrors real utility company departments. CONSUMER can only see their own bills and payments. BILLING executives generate and manage bills but cannot touch outage data. FIELD_TECH handles meter installation and field dispatch. GRID_OPS manages outage events and field operations. COMPLIANCE views audit reports for regulatory purposes. ADMIN has full system access. This follows the principle of least privilege — each user gets only the access they need to do their job, reducing security risk.

Q3. How does JWT authentication work end-to-end in your project?

A: The client sends POST /api/auth/login with email and password in the request body. AuthService loads the user by email using UserRepository.findByEmail(). It verifies the password using BCryptPasswordEncoder.matches(rawPassword, storedHash). On success, JwtUtil.generateToken() builds a JWT: header = {alg: HS256, typ: JWT}, payload = {sub: email, role: ADMIN, iat: now, exp: now + 24h}, signature = HMAC-SHA256(header.payload, secretKey). Response returns {token, role, userId}. For every subsequent request, JwtFilter (extends OncePerRequestFilter) runs before the controller: it extracts the Bearer token, validates signature and expiry via JwtUtil, extracts email and role, and sets a UsernamePasswordAuthenticationToken into SecurityContextHolder so Spring Security treats the request as authenticated.

Q4. What is BCrypt and why do you use it for passwords?

A: BCrypt is an adaptive password-hashing algorithm that incorporates a random salt and a configurable cost factor (default 10 in Spring). It is deliberately slow — making brute-force attacks computationally expensive. Spring Security provides BCryptPasswordEncoder. On user creation: `encoder.encode(rawPassword)` generates a unique hash. On login: `encoder.matches(rawPassword, storedHash)` safely compares without exposing the original. I never store plain text passwords. The cost factor can be increased as hardware improves — BCrypt is future-proof.

Q5. What does @Enumerated(EnumType.STRING) do and why is it important?

A: `@Enumerated(EnumType.STRING)` tells Hibernate to store the enum constant's name as a VARCHAR string in the database column (e.g., 'CONSUMER', 'ADMIN', 'ACTIVE') instead of its ordinal integer position (0, 1, 2...). This is critically important for data safety: if you ever reorder enum constants, ordinal-based storage would corrupt all existing records because the integers would map to different values. STRING storage is safe because it is tied to the constant name. All 20+ enums in EnergyGrid use EnumType.STRING.

Q6. Explain AuditLogService and why it is designed as a shared method.

A: AuditLogService has one public method: `logAction(Integer userId, String action, String entityType, Integer entityId)`. Every other service calls this method after significant mutations — for example, BillingService calls `logAction` after generating a bill, PaymentService after recording a payment, OutageEventService after restoring an outage. By centralising this in a separate @Service bean, I avoid duplicating audit logging code across 10+ services. The AOP LoggingAspect explicitly excludes AuditLogServiceImpl from its pointcut expression to prevent recursive logging loops where logging an audit entry would trigger another audit log entry.

Q7. How does EnergyReportService generate its reports and what does the Metrics field contain?

A: `EnergyReportService.generateReport(scope)` queries multiple repositories based on the scope. For GRID scope it counts all consumer accounts, calculates total bill amounts vs payments received to compute a collection rate, fetches active outage count from OutageEventRepository, and sums meter reading values for MWh distributed. It builds a JSON string — for example: `{totalConsumers: 12450, collectionRate: 98.5, activeOutages: 3, mwhDistributed: 450}` — and stores it in the Metrics TEXT column. The GET `/api/energy-reports/summary` endpoint reads these records and returns the KPI values for the analytics dashboard front end.

Q8. What JUnit tests did you write for UserService and what do they verify?

A: I wrote 5 tests with `@ExtendWith(MockitoExtension.class)`. (1) `createUser_validInput_returnsSavedUser`: mock `UserRepository.save` returns user, assert returned DTO has correct name and role. (2) `createUser_duplicateEmail_throwsRuntimeException`: mock `findByEmail` returns an existing user, assert `RuntimeException` is thrown. (3) `getUserById_validId_returnsUser`: mock `findById` returns `Optional.of(user)`, assert user is returned. (4) `getUserById_invalidId_throwsResourceNotFoundException`: mock `findById` returns `Optional.empty()`, assert `ResourceNotFoundException`. (5) `updateUserStatus_validId_changesStatus`: mock `findById`, call `updateUserStatus`, assert status field changed and save was called. All 5 tests pass with mvn test.

Q9. What does SecurityConfig do and what are its key settings?

A: SecurityConfig is a `@Configuration @EnableWebSecurity` class that defines a SecurityFilterChain bean. Key settings: (1) `csrf.disable()` — CSRF protection turned off because this is a stateless REST API with no sessions — the JWT itself provides the protection. (2) `sessionManagement.sessionCreationPolicy(STATELESS)` — no HTTP sessions created. (3) `authorizeHttpRequests`: permits all requests to `/api/auth/**`, requires authentication for all other paths. (4) `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)` — JwtFilter runs before Spring's default form-login filter. (5) cors configuration allows origin localhost:3000 so the React frontend can make API calls.

Q10. Explain the custom query methods in UserRepository and AuditLogRepository.

A: UserRepository extends JpaRepository and declares: `findByEmail(String email)` returning `Optional[User]` — used by AuthService for login lookup; `findByStatus(UserStatus)` returning `List[User]` — used for admin user management. AuditLogRepository declares: `findById(Integer userId)`, `findByAction(String action)`, `findByTimestampBetween(LocalDateTime from, LocalDateTime to)`, and `findAll(Pageable pageable)` inherited from JpaRepository for paginated audit log queries. Spring Data JPA auto-generates SQL from method names at startup — no SQL written manually.

Q11. Explain how AuthContext works and why you need it.

A: AuthContext.jsx creates a React Context object using createContext(). The AuthProvider component wraps the entire App and provides: user state (containing {userId, role, token}), a login() function that calls POST /api/auth/login, stores the response in localStorage, and updates state, and a logout() function that clears localStorage and resets state. A custom hook useAuth() wraps useContext(AuthContext) for convenient access in any component. Without Context, every component would need userId and role passed down as props through multiple levels (prop drilling), which becomes unmanageable.

Q12. What is ProtectedRoute and why is it necessary?

A: ProtectedRoute.jsx is a wrapper component that checks whether a JWT token exists in localStorage. If the token is absent, it uses React Router's Navigate component to redirect to /login. If the token is present, it renders the wrapped child component. In App.jsx every route except /login is wrapped: the Billing, Consumer, and all other pages require authentication. Without ProtectedRoute, anyone who knows the URL /bills could access that page directly without logging in.

Q13. How did you implement role-based sidebar navigation?

A: Layout.jsx calls useAuth() to get the current user's role from context. I defined a navConfig object: an array of entries where each entry has a label, path, and an array of allowed roles. The sidebar maps over navConfig and filters out items where allowedRoles does not include the current role. For example, CONSUMER sees only Bills and Payments. BILLING sees Tariffs, Bills, and Disputes. ADMIN sees everything. This is UI-level protection — the backend JWT and endpoint authorization provide the real security.

Q14. How are the KPI cards in AnalyticsDashboard populated?

A: AnalyticsDashboard has 4 Bootstrap Card components: Total Consumers, Collection Rate percentage, Active Outages, and MWh Distributed. In Phase 1 (Wk9-10) these showed hardcoded mock values stored in useState for static page development. In Phase 3 (Wk13 API integration), a useEffect hook with empty dependency array runs on component mount. It calls reportService.getSummary() which sends GET /api/energy-reports/summary. The response data populates the state variables. While the API call is pending, a Bootstrap Spinner is rendered in place of each card value.

Q15. Explain how the axios interceptor works in api.js.

A: In src/services/api.js, I create an axios instance: axios.create({baseUrl: 'http://localhost:8095'}). I then add a request interceptor using api.interceptors.request.use(). The interceptor function reads the token from localStorage. If a token exists, it sets config.headers.Authorization = 'Bearer ' + token on the outgoing request config object, then returns the config. This means every axios call made through this instance automatically includes the JWT header — individual service files (userService.js, reportService.js) do not need to manually add the Authorization header each time.

HOW TO EXPLAIN YOUR MODULE TO THE INTERVIEWER

Backend — Billing and Tariff Management: I built the complete billing engine for EnergyGrid. My module covers three tables: tariffs, bills, and billing_disputes. The Tariff entity stores slab-based pricing per consumer category — RESIDENTIAL, COMMERCIAL, or INDUSTRIAL. The slabRates field is stored as a JSON TEXT string containing an array of slab ranges with rates (e.g., 0-100 units at Rs.3.50, 101-300 at Rs.5.00, above 300 at Rs.7.50). BillingService.generateBill() fetches the tariff, parses the JSON slabs, applies the correct rate to units consumed, adds fixed charges, then applies tax to get the total amount. There is a batch endpoint PATCH /api/bills/mark-overdue that finds all GENERATED bills past their DueDate and marks them OVERDUE. BillingDispute has a four-state lifecycle: OPEN, UNDERREVIEW, RESOLVED, REJECTED. All services use @Slf4j for logging.

Frontend — Billing Dashboard + Dispute UI: TariffListPage shows a Bootstrap table with all tariffs. BillListPage has four summary cards at the top showing Total Bills, Paid count, Overdue count, and Disputed count — computed from the bills array in state using JavaScript filter and reduce. GenerateBillModal is a controlled form where selecting a tariff and entering units auto-calculates the estimated amount client-side as a preview. BillingDisputePage lists open disputes with Resolve and Reject buttons. Status badges use Bootstrap color classes: PAID=green, OVERDUE=red, DISPUTED=yellow.

BACKEND INTERVIEW QUESTIONS — Billing & Tariff Management**Q1. Explain the Tariff entity structure and the slabRates field in detail.**

A: Tariff entity: TariffID (PK), Name (VARCHAR 150), ConsumerCategory (ENUM: RESIDENTIAL / COMMERCIAL / INDUSTRIAL stored as STRING), SlabRates (TEXT — stores JSON like [{from:0,to:100,rate:3.5},{from:101,to:300,rate:5.0},{from:301,to:999999,rate:7.5}]), FixedCharges (DECIMAL 10,2), TaxRates (DECIMAL 5,2), EffectiveDate (DATE), Status (ENUM: ACTIVE / EXPIRED DEFAULT ACTIVE). I chose to store slabRates as a JSON string in a TEXT column rather than creating a separate TariffSlab table because: the number of slabs varies per category, the slabs are always read and written as a unit, and a TEXT JSON avoids a JOIN query on every bill generation. BillingService uses Jackson ObjectMapper to parse the JSON at runtime.

Q2. Walk me through how generateBill() calculates TotalAmount.

A: BillingService.generateBill(accountId, unitsConsumed, tariffId): (1) Fetch Tariff by tariffId — throw ResourceNotFoundException if not found. (2) Parse slabRates JSON string using ObjectMapper.readValue() into a List of SlabDTO objects. (3) Iterate slabs to find the matching rate: the slab where unitsConsumed falls between slab.from and slab.to. (4) slabAmount = unitsConsumed multiplied by the matching rate. (5) totalAmount = (slabAmount + tariff.fixedCharges) multiplied by (1 + tariff.taxRates / 100). (6) Create a Bill entity with accountId, billingPeriodStart/End, unitsConsumed, tariff reference, totalAmount, dueDate (30 days from today), status=GENERATED. (7) Save and return. Call AuditLogService.logAction() after save.

Q3. What does PATCH /api/bills/mark-overdue do and how is it implemented?

A: This is a batch update endpoint called by a Billing Executive or a scheduler. BillingService.markOverdue() calls BillRepository.findByStatus(BillStatus.GENERATED) to get all generated bills. It then filters in Java where bill.getDueDate().isBefore(LocalDate.now()). For each expired bill it calls bill.setStatus(BillStatus.OVERDUE) then billRepository.saveAll(overdueList). The JUnit test mockOverdue_updatesExpiredBills mocks the repository to return 2 expired bills and asserts both are saved with OVERDUE status. The controller returns HTTP 200 with the count of bills marked overdue.

Q4. Describe the complete BillingDispute lifecycle.

A: Step 1 — Consumer raises dispute: POST /api/billing-disputes with billId, consumerId, reason. DisputeService.raiseDispute() creates entity with status=OPEN, raisedDate=CURDATE(). Step 2 — Billing exec reviews: PATCH /api/billing-disputes/{id}/status?status=UNDERREVIEW. Step 3a — Resolved: PATCH /api/billing-disputes/{id}/resolve sets status=RESOLVED. Step 3b — Rejected: PATCH /api/billing-disputes/{id}/reject sets status=REJECTED. Business rule enforced in service: if dispute.status is already RESOLVED or REJECTED, throw

BadRequestException("Dispute already closed"). The JUnit test `resolveDispute_alreadyResolved_throwsException` verifies this rule.

Q5. Explain @ManyToOne and @JoinColumn in your Bill entity.

A: Bill.java has: `@ManyToOne @JoinColumn(name='TariffID')` private Tariff tariff. `@ManyToOne` means many Bill records can reference the same Tariff — the Bills table is the owning side of the relationship.

`@JoinColumn(name='TariffID')` tells Hibernate to create a column named `TariffID` in the bills table that is a foreign key pointing to `tariffs.TariffID`. When I load a Bill, Hibernate automatically executes a JOIN to populate the tariff field (EAGER loading — the default for `@ManyToOne`). This lets me access `bill.getTariff().getFixedCharges()` directly in service code.

Q6. Why does BillingDispute store consumerId as Integer instead of @ManyToOne to User?

A: This is a deliberate loose coupling design decision following bounded context principles. The Billing module should not have a hard JPA dependency on the IAM module's User entity. If the project were later decomposed into microservices, the Billing service would not need a direct database link to the Users table. The `consumerId` Integer stores the reference logically. Validation — checking that the consumer exists — can be done through a service call rather than a JPA join. This same pattern is used for `accountId` in Bill and `installmentPlan` — intentionally no FK constraints in those cases.

Q7. What are your BillRepository custom query methods?

A: `BillRepository` extends `JpaRepository` and declares: `findByAccountId(Integer accountId)` returning `List[Bill]` — bills for a specific consumer account; `findByStatus(BillStatus status)` returning `List[Bill]` — used by mark-overdue; `findByTariff_TariffId(Integer tariffId)` returning `List[Bill]` — bills using a specific tariff; `findByAccountIdAndStatus(Integer, BillStatus)` — bills for account filtered by status. The `findByTariff_TariffId` method uses underscore traversal: `Tariff` is the `@ManyToOne` field name, `TariffId` is the field within `Tariff` — Spring Data navigates the relationship.

Q8. What JUnit tests did you write for BillingService?

A: `BillingServiceTest` has 4 tests: (1) `generateBill_calculatesCorrectAmount`: mock tariff with `fixedCharge=50`, `taxRate=10`, `units=150` falling in slab `rate=5.0`, assert `totalAmount = (150x5.0+50) x 1.10 = 880`. (2) `getBillsByAccount_returnsCorrectList`: mock repository returns 3 bills for `accountId=1`, assert list size=3. (3) `updateBillStatus_fromGeneratedToPaid`: mock `findById`, assert status changed and save called. (4) `markOverdue_updatesAllExpiredBills`: mock `findByStatus` returns 2 `GENERATED` expired bills, assert both saved as `OVERDUE`. `BillingDisputeServiceTest` has 3 tests including the `alreadyResolved` exception test. All 7 pass with `@Slf4j` verified in all services.

FRONTEND INTERVIEW QUESTIONS — Billing Dashboard + Dispute UI

Q9. How does GenerateBillModal auto-calculate the amount as the user fills in the form?

A: `GenerateBillModal.jsx` is a controlled component. It has state variables: `accountId`, `unitsConsumed` (number input), `selectedTariff` (from a select dropdown populated from the tariffs list), and `calculatedAmount`. An `onChange` handler on both the units input and the tariff select calls `recalculate()`. `recalculate()` parses the `selectedTariff`'s `slabRates` JSON, finds the matching slab for the entered units, computes `slabAmount + fixedCharges`, applies tax, and sets `calculatedAmount` state. The form shows this preview amount. The actual authoritative calculation happens server-side in `BillingService` when `POST /api/bills` is called on submit.

Q10. Explain how the summary cards in BillListPage work.

A: `BillListPage` fetches all bills for the selected account into a bills array in `useState`. The four summary Bootstrap Cards at the top are computed directly from this array: `Total Bills = bills.length`. `Paid = bills.filter(b => b.status === 'PAID').length`. `Overdue = bills.filter(b => b.status === 'OVERDUE').length`. `Disputed = bills.filter(b => b.status === 'DISPUTED').length`. These are pure JavaScript computations — no extra API calls. They update reactively whenever the bills state changes, for example after a payment is recorded.

Q11. How do status badges work for bills and disputes?

A: I created a helper function `getStatusVariant(status)` that returns Bootstrap CSS class names: `PAID` maps to badge `bg-success`, `OVERDUE` maps to badge `bg-danger`, `GENERATED` maps to badge `bg-secondary`, `SENT` maps to badge `bg-primary`, `DISPUTED` maps to badge `bg-warning text-dark`, `OPEN` maps to badge `bg-warning text-dark`,

RESOLVED maps to badge bg-success, REJECTED maps to badge bg-danger. Each table row renders a span element with className set to the returned string and the status text as its content. This gives visual color coding without any custom CSS.

Q12. How does the Resolve Dispute button work?

A: In BillingDisputePage, each OPEN dispute row has a Resolve button and a Reject button. Clicking Resolve calls `window.confirm()` with a message asking the user to confirm. If confirmed, it calls `disputeService.resolveDispute(disputeId)` which sends `PATCH /api/billing-disputes/{id}/resolve`. On a successful response, I update local state: `setDisputes(prev => prev.map(d => d.id === disputeId ? {...d, status:'RESOLVED'} : d))`. This is an optimistic update — the UI reflects the change immediately without re-fetching the entire list. If the API returns an error (e.g., already resolved), an error alert is shown.

HOW TO EXPLAIN YOUR MODULE TO THE INTERVIEWER

Backend — Consumer Account + Connection + Meter Data: My module manages the physical and logical lifecycle of a utility consumer. I have four entities: ConsumerAccount (the master record for each consumer's service connection — stores address, connection type, zone, meter reference, and status), ConnectionRequest (when a consumer requests a new connection, upgrade, disconnection, or reconnection — goes through an assign-technician workflow), Meter (physical device linked to an account — types: ANALOG, DIGITAL, SMART_AMI), and MeterReading (actual consumption readings — source can be MANUAL, AMI, or ESTIMATED). I have 4 entities, 4 repositories, 4 services, and 8 enums. The calculateConsumption() method uses the earliest and latest readings in a date range to compute kWh consumed. findTop1ByMeter_MeterIdOrderByReadingDateDesc() is my most complex derived query.

Frontend — Consumer Portal + Meter Reading UI: ConsumerListPage with Bootstrap table and account status badges. ConnectionRequestPage with an assign-technician workflow where clicking Assign opens a dropdown of FIELD_TECH users. MeterReadingPage has a meter selector dropdown — on selection it fetches all readings and calculates consumption as (latest reading value minus previous reading value) using plain JavaScript.

BACKEND INTERVIEW QUESTIONS — Consumer Account + Meter Data**Q1. Explain your four database tables and their relationships.**

A: ConsumerAccount: AccountID(PK), ConsumerID(INT no FK — loose coupling), ServiceAddress(TEXT), ConnectionType(ENUM: RESIDENTIAL/COMMERCIAL/INDUSTRIAL), ZoneID(INT), MeterID(INT no FK), Status(ENUM: ACTIVE/SUSPENDED/DISCONNECTED). ConnectionRequest: RequestID(PK), ConsumerID(INT), RequestType(ENUM: NEWCONNECTION/UPGRADE/DISCONNECTION/RECONNECTION), RequestDate(DEFAULT CURDATE()), AssignedTechID(INT no FK), Status(ENUM: PENDING/SCHEDULED/COMPLETED/REJECTED). Meter: MeterID(PK), AccountID(FK to consumer_accounts), MeterNumber(VARCHAR 50 UNIQUE), Type(ENUM: ANALOG/DIGITAL/SMARTAMI), InstalledDate(DATE), Status(ENUM: ACTIVE/FAULTY/REPLACED). MeterReading: ReadingID(PK), MeterID(FK to meters), ReadingValue(DECIMAL 10,2), ReadingDate(DATE), Source(ENUM: MANUAL/AMI/ESTIMATED), Status(ENUM: VALIDATED/ESTIMATED/DISPUTED).

Q2. How does the ConsumerAccount relate to the Meter entity in JPA?

A: ConsumerAccount has a meterId field (Integer — no @ManyToOne) that stores the ID of the current active meter. The Meter entity is on the owning side with @ManyToOne @JoinColumn(name='AccountID') ConsumerAccount account — so one account can have multiple meters over time (the old one gets status=REPLACED, a new one gets status=ACTIVE). When a meter is replaced, MeterService.installMeter() creates the new Meter linked to the account, then calls ConsumerAccountService to update consumerAccount.setMeterId(newMeter.getId()). This design preserves meter replacement history.

Q3. Explain MeterReadingService.calculateConsumption() in detail.

A: calculateConsumption(Integer meterId, LocalDate from, LocalDate to) calls MeterReadingRepository.findByMeter_MeterIdAndReadingDateBetween(meterId, from, to) to get all readings in the date range. It then sorts the list by readingDate. The first element (earliest) is the starting reading; the last element (latest) is the ending reading. Consumption in kWh = lastReading.readingValue minus firstReading.readingValue. This value is returned and used by BillingService as unitsConsumed when generating a bill. If fewer than 2 readings exist, it throws a BadRequestException('Insufficient readings for consumption calculation').

Q4. What is findTop1ByMeter_MeterIdOrderByReadingDateDesc() and how does Spring Data handle it?

A: This is a Spring Data JPA derived query method declared in MeterReadingRepository. Spring Data parses the method name: findTop1 means add LIMIT 1 to the SQL query; ByMeter_MeterId means traverse the @ManyToOne meter field to filter on meter.meterId; OrderByReadingDateDesc means add ORDER BY reading_date DESC. Generated SQL: SELECT * FROM meter_readings WHERE meter_id = ? ORDER BY reading_date DESC LIMIT 1. This returns the single most recent reading for a given meter. I use it in getLatestReading(meterId) which is called by

the frontend to show the current meter reading and by the consumption calculation fallback.

Q5. Explain the ConnectionRequest workflow step by step.

A: Step 1: Consumer calls POST /api/connection-requests with requestType and address. ConnectionRequestService.createRequest() saves entity with status=PENDING, requestDate=today. Step 2: Admin calls PATCH /api/connection-requests/{id}/assign?techId=5. assignTechnician() fetches the request, validates that the techId corresponds to a real user with FIELD_TECH role, sets assignedTechId=5, status=SCHEDULED, saves. Step 3: Field tech completes work. Admin calls PATCH /api/connection-requests/{id}/status?status=COMPLETED. Alternatively PATCH /api/connection-requests/{id}/status?status=REJECTED if not feasible. MeterService.installMeter() is called separately after COMPLETED to link the meter.

Q6. What are the 8 enums in your module and why so many?

A: ConnectionType (RESIDENTIAL, COMMERCIAL, INDUSTRIAL) — type of service connection. AccountStatus (ACTIVE, SUSPENDED, DISCONNECTED) — current connection status. RequestType (NEWCONNECTION, UPGRADE, DISCONNECTION, RECONNECTION) — type of request. RequestStatus (PENDING, SCHEDULED, COMPLETED, REJECTED) — request lifecycle. MeterType (ANALOG, DIGITAL, SMARTAMI) — SMART_AMI meters auto-transmit readings. MeterStatus (ACTIVE, FAULTY, REPLACED) — physical device status. ReadingSource (MANUAL, AMI, ESTIMATED) — how reading was obtained. ReadingStatus (VALIDATED, ESTIMATED, DISPUTED) — accuracy status. Using enums prevents invalid string values, enables type-safe comparisons in services, and makes the domain model self-documenting.

Q7. What does MeterService.installMeter() validate before saving?

A: installMeter() performs three validations: (1) accountId validation: calls ConsumerAccountRepository.findById(accountId) — if Optional is empty, throws ResourceNotFoundException('Account not found: ' + accountId). (2) Duplicate meterNumber check: calls MeterRepository.findByMeterNumber(meterNumber) — if present, throws BadRequestException('Meter number already exists'). (3) Account status check: if account.status == SUSPENDED or DISCONNECTED, throws BadRequestException('Cannot install meter for inactive account'). Only after all validations pass does it create the Meter entity, set status=ACTIVE, save it, and call AuditLogService.logAction().

Q8. What JUnit tests did you write for ConsumerAccountService and MeterReadingService?

A: ConsumerAccountServiceTest: (1) registerConsumer_validInput_savedWithActiveStatus: mock save, assert status=ACTIVE. (2) getAccountById_invalidId_throwsResourceNotFoundException: mock findById empty. (3) updateAccountStatus_validId_changesStatus: mock findById, assert status changed. MeterReadingServiceTest: (1) recordReading_validInput_savedWithValidatedStatus: mock save, assert status=VALIDATED. (2) calculateConsumption_twoReadings_returnsCorrectUnits: mock returns 2 readings with values 1000 and 1150, assert consumption = 150.0 kWh. (3) getLatestReading_meterId_returnsMostRecent: mock findTop1 returns reading. Total 6 tests, all pass. @Slf4j verified in all 4 service classes.

FRONTEND INTERVIEW QUESTIONS — Consumer Portal + Meter Reading UI

Q9. How does consumption calculation work in MeterReadingPage?

A: MeterReadingPage has a meter selector dropdown. When the user selects a meter, an onChange handler calls meterReadingService.getReadingsByMeter(meterId) which sends GET /api/meter-readings?meterId=X. The response array is sorted by readingDate descending and stored in useState. Below the readings table, the component renders: const consumption = readings.length >= 2 ? (readings[0].readingValue - readings[1].readingValue).toFixed(2) : 'N/A'. This is a pure JavaScript calculation on the client — no extra API call. The result is displayed as 'Current Consumption: X kWh'.

Q10. How did you implement the Assign Technician button in ConnectionRequestPage?

A: Each row in the connection requests table that has status=PENDING shows an Assign button. Clicking it sets a selectedRequestId in useState and shows a dropdown or modal. The dropdown is populated from userService getUsersByRole('FIELD_TECH') which fetches GET /api/users?role=FIELD_TECH. When a technician is selected and submitted, it calls connectionRequestService.assignTechnician(requestId, techId) which sends PATCH /api/connection-requests/{requestId}/assign?techId={techId}. On success, state is updated: setRequests(prev => prev.map(r => r.id === requestId ? {...r, status:'SCHEDULED', assignedTechId: techId} : r)).

HOW TO EXPLAIN YOUR MODULE TO THE INTERVIEWER

Backend — Payments and Collections + Notifications: My module handles money flow and in-app communications. The Payment entity records each payment transaction — BillID reference (loose coupling, Integer not FK), AccountID, Amount, PaymentDate, Method (ONLINE/CASH/CHEQUE/AUTODEBIT), and Status (PENDING/RECEIVED/FAILED). PaymentService is marked @Transactional because recordPayment() does two things atomically: it saves the Payment record with status=RECEIVED AND updates the referenced Bill status to PAID. If either operation fails, both roll back. It also validates that the bill is not already PAID before processing, returning HTTP 400 if so. InstallmentPlan allows large overdue amounts to be paid in multiple instalments (ACTIVE/COMPLETED/DEFAULTED). NotificationService manages in-app messages with 5 categories and tracks unread counts using countByUserIdAndStatus().

Frontend — Payment Portal + Notifications: PaymentListPage with Bootstrap table and method/status badges. RecordPaymentModal form. InstallmentPlanPage with complete/default buttons. NotificationsPage with 6 category filter tabs (All, BILLING, PAYMENT, OUTAGE, METER, COMPLIANCE) — filtered client-side from a single fetched array. NotificationBell component in Navbar polls getUnreadCount() every 30 seconds using setInterval — the cleanup function in useEffect calls clearInterval to prevent memory leaks on unmount.

BACKEND INTERVIEW QUESTIONS — Payments + Notifications**Q1. Why is recordPayment() annotated with @Transactional and what happens without it?**

A: @Transactional wraps the entire method in a single database transaction. Inside recordPayment(), I do two operations: (1) paymentRepository.save(payment) — create the payment record. (2) billRepository.findById(billId) followed by bill.setStatus(PAID) and billRepository.save(bill). If operation 2 fails (for example, the bill ID does not exist), @Transactional automatically rolls back operation 1 as well — the database remains consistent, no orphan payment record. Without @Transactional, each save() call commits independently — a failure in step 2 would leave a payment recorded in the database but the bill status would remain GENERATED, causing a data inconsistency that would confuse consumers.

Q2. What validations does POST /api/payments perform?

A: PaymentController passes the PaymentRequest to PaymentService.recordPayment(). Validations in order: (1) Fetch the bill: billRepository.findById(billId) — if Optional is empty, throw ResourceNotFoundException('Bill not found'). (2) Duplicate payment check: if bill.getStatus() == BillStatus.PAID, throw BadRequestException('Bill is already paid') — controller returns HTTP 400. (3) Amount validation: if request.getAmount() <= 0, throw BadRequestException. (4) On all validations passing: save Payment with status=RECEIVED, update bill.status=PAID, save bill, call AuditLogService, send PAYMENT notification to the consumer.

Q3. Explain the InstallmentPlan use case and lifecycle.

A: A consumer has a large overdue bill of Rs.15,000 that they cannot pay at once. An admin or billing executive creates an InstallmentPlan: POST /api/installment-plans with accountId, totalDue=15000, installments=3, startDate, endDate. Status starts as ACTIVE. The consumer makes 3 payments of Rs.5,000 each — each recorded via POST /api/payments. When all instalments are paid, the exec calls PATCH /api/installment-plans/{id}/complete to set status=COMPLETED. If the consumer misses payments past endDate: PATCH /{id}/default sets status=DEFAULTED, which may trigger a collections escalation workflow or legal action notification.

Q4. Describe all NotificationService methods and when they are called.

A: sendNotification(userId, message, category): creates Notification with status=UNREAD, createdAt=now(). Called by BillingService after bill generation (category=BILLING), by PaymentService after payment (PAYMENT), by OutageEventService after new outage in affected zone (OUTAGE), by MeterService after faulty meter detected (METER), by EnergyReportService for compliance reports (COMPLIANCE). getByUser(userId): returns list ordered by createdAt DESC (latest first). markAsRead(id): sets status=READ. markAllAsRead(userId): calls repository to update all UNREAD for that user, returns count. getUnreadCount(userId): calls countByUserIdAndStatus(userId, UNREAD) — efficient COUNT query.

Q5. What is countByUserIdAndStatus() and how does Spring Data generate it?

A: NotificationRepository declares: Long countByUserIdAndStatus(Integer userId, NotificationStatus status). Spring Data JPA generates: SELECT COUNT(*) FROM notifications WHERE user_id = ? AND status = ? This is more efficient than findById() and calling .size() on the returned list because it executes a single COUNT query — no need to load all notification objects into memory. The GET /api/notifications/user/{userId}/count endpoint calls this and returns {unreadCount: N} — used by the frontend notification bell badge.

Q6. Explain PATCH /api/notifications/user/{userId}/read-all and its response format.

A: NotificationController.markAllAsRead(userId) calls NotificationService.markAllAsRead(userId). The service calls NotificationRepository.findByIdAndStatus(userId, UNREAD) to get all unread notifications for that user. It then sets each one's status to READ and calls notificationRepository.saveAll(list). The service returns the count of updated records (int). The controller wraps this in a Map: Map.of('updated', count) and returns ResponseEntity.ok(map). The JSON response is {updated: 4} — the frontend uses this count to display a toast message like '4 notifications marked as read'.

Q7. What JUnit tests did you write for PaymentService and NotificationService?

A: PaymentServiceTest: (1) recordPayment_validBill_savesPaymentAndUpdatesBillToPaid: mock billRepo returns GENERATED bill, assert paymentRepo.save called, assert bill.status changed to PAID. (2) recordPayment_billAlreadyPaid_throwsBadRequestException: mock billRepo returns PAID bill. (3) getPaymentsByAccount_returnsCorrectList: mock returns 3 payments for accountId. NotificationServiceTest: (1) sendNotification_createdWithUnreadStatus: assert status=UNREAD after save. (2) markAsRead_changesStatusToRead: mock findById returns UNREAD, assert READ after call. (3) getUnreadCount_callsCountMethod: mock countByUserIdAndStatus returns 5, assert result=5. (4) markAllAsRead_updatesAllAndReturnsCount: mock returns 3 UNREAD, assert count=3. Total 7 tests, all pass with mvn test.

FRONTEND INTERVIEW QUESTIONS — Payment Portal + Notifications UI**Q8. How does NotificationBell polling work and why is cleanup important?**

A: NotificationBell.jsx is a component rendered inside the Navbar. It has an unreadCount in useState initialised to 0. A useEffect with empty dependency array runs on mount: it fetches the initial count, then sets up setInterval with a 30,000 ms interval calling notificationService.getUnreadCount(userId) and updating state. The critical part: the useEffect returns a cleanup function: return () => clearInterval(intervalId). This cleanup runs when the component unmounts (e.g., user logs out). Without clearInterval, the interval would keep firing even after unmount, causing setState calls on an unmounted component — a React memory leak warning and potential errors.

Q9. How do the category filter tabs in NotificationsPage work?

A: NotificationsPage fetches all notifications for the user once on mount via notificationService.getByUser(userId), storing them in a notifications array in useState. There are 6 Bootstrap Nav tabs: All, BILLING, PAYMENT, OUTAGE, METER, COMPLIANCE. Clicking a tab sets activeCategory in useState. The rendered list is a derived value: const displayed = activeCategory === 'All' ? notifications : notifications.filter(n => n.category === activeCategory). This is pure client-side filtering — no API call on tab switch. The page loads once and the user can filter freely without network requests.

Q10. How does the Mark All Read button update the UI?

A: Clicking Mark All Read calls notificationService.markAllAsRead(userId) which sends PATCH /api/notifications/user/{userId}/read-all. The API returns {updated: N}. On successful response, I update local state using: setNotifications(prev => prev.map(n => ({...n, status: 'READ'}))). This spread operator creates new objects for each notification with status overridden to READ — React detects the state change and re-renders. The bell badge also resets to 0 by setting unreadCount state to 0. This optimistic local update avoids a full re-fetch from the server.

HOW TO EXPLAIN YOUR MODULE TO THE INTERVIEWER

Backend — Outage and Grid Event Management: My module manages the complete lifecycle of power outages and field operations. OutageEvent captures zone-level outages: ZoneID (which zone is affected), AffectedAccounts (number of consumers impacted), Cause (PLANNED/UNPLANNED/WEATHER/EQUIPMENT), StartTime, EstimatedRestoration (the SLA commitment), ActualRestoration (set when the outage is resolved), and Status (ACTIVE/RESTORED/INVESTIGATING). FieldDispatch links a technician to an outage and tracks their on-ground progression: DISPATCHED when sent, ONSITE when they arrive, RESOLVED when they fix the issue. The SLA endpoint calculates whether the actual restoration time was within the estimated commitment. I wrote the most tests: 7 JUnit tests covering both services.

Frontend — Grid Ops Dashboard + Outage Tracker: OutageListPage shows outages with ACTIVE in red and RESTORED in green badges. Each ACTIVE row has a Restore button that calls window.confirm before sending the API request. FieldDispatchPage has contextual status buttons: DISPATCHED rows show Mark On-Site, ONSITE rows show Mark Resolved, RESOLVED rows show only the badge. A summary row at the top shows Active Outages count, Total Affected Accounts (sum), and Average Resolution Time — all computed in JavaScript.

BACKEND INTERVIEW QUESTIONS — Outage & Grid Event Management

Q1. Explain all fields in OutageEvent and their purpose.

A: OutageID (PK, auto-increment). ZoneID (INT — identifies geographic service zone). AffectedAccounts (INT — estimated number of consumers without power). Cause (ENUM: PLANNED=scheduled maintenance, UNPLANNED=unexpected failure, WEATHER=storm/lightning, EQUIPMENT=transformer/cable failure). StartTime (DATETIME — when outage began). EstimatedRestoration (DATETIME — committed restoration time communicated to consumers). ActualRestoration (DATETIME — null until restoreOutage() is called, then set to now()). Status (ENUM: ACTIVE=currently without power, RESTORED=power restored, INVESTIGATING=cause being determined). All time fields use LocalDateTime in Java, mapped to DATETIME in MySQL.

Q2. How does the SLA check endpoint work?

A: GET /api/outage-events/{id}/sla calls OutageEventService.checkSla(id). It fetches the OutageEvent by ID. It builds a response object (or Map) with three fields: estimatedRestoration (the original commitment), actualRestoration (when power was actually restored — may be null if still ACTIVE), and slaMet (boolean). slaMet calculation: if actualRestoration is null, slaMet = false (outage not yet resolved). Otherwise: slaMet = !actualRestoration.isAfter(estimatedRestoration) — true if actual was on time or early, false if late. This allows grid operations analysts to report on service level agreement performance across all outages in a period.

Q3. Why is FieldDispatchService @Transactional and what does dispatchTechnician() validate?

A: @Transactional ensures that creating a FieldDispatch record and any associated updates are atomic. dispatchTechnician(outageId, technicianId, dispatchTime) validates: (1) OutageEvent exists: outageEventRepository.findById(outageId) — throws ResourceNotFoundException if empty. (2) Outage is actionable: if outageEvent.status == RESTORED, throw BadRequestException('Cannot dispatch to an already restored outage'). (3) After validation, creates FieldDispatch entity with status=DISPATCHED, dispatchTime=now (using @CreationTimestamp). Saves and logs audit. Returns HTTP 201 on success, HTTP 400 on invalid outageId.

Q4. Explain restoreOutage() and its business rule.

A: OutageEventService.restoreOutage(id): (1) Fetch OutageEvent by id — throw ResourceNotFoundException if not found. (2) Business rule check: if outageEvent.status == RESTORED, throw BadRequestException('Outage already restored') — you cannot restore an already-restored outage. Controller returns HTTP 400 in this case. (3) If ACTIVE or INVESTIGATING: set outageEvent.status = RESTORED, set outageEvent.actualRestoration = LocalDateTime.now(), save. (4) The actualRestoration timestamp is then used by checkSla() to determine if the restoration was within the estimated time commitment. Controller returns HTTP 200 with the updated OutageEvent.

Q5. What is findByStartTimeBetween() and where is it used?

A: OutageEventRepository declares: List<OutageEvent> findByStartTimeBetween(LocalDate from, LocalDate to). Spring Data generates: SELECT * FROM outage_events WHERE start_time BETWEEN ? AND ?. This is used by EnergyReportService when generating PERIOD-scoped energy reports — it fetches all outages that started within the reporting period to calculate metrics like total outages, average duration, zones most affected, and total affected accounts. It is also used in the Grid Ops analytics dashboard to show outage trends over a date range selected by the user.

Q6. Explain FieldDispatch status progression and resolveDispatch().

A: FieldDispatch status flow: DISPATCHED (initial — technician assigned and sent) to ONSITE (technician has arrived at location) to RESOLVED (issue fixed). updateDispatchStatus(id, newStatus) validates the status transition is forward-only. resolveDispatch(id) is a dedicated endpoint: it fetches the FieldDispatch, validates status is not already RESOLVED, sets status=RESOLVED and resolutionTime=LocalDateTime.now(). The resolutionTime minus dispatchTime gives the total field response time — used for technician performance reporting. @Transactional ensures the status and timestamp update are atomic.

Q7. What are all 7 JUnit tests you wrote?

A: OutageEventServiceTest (4 tests): (1) reportOutage_savedWithActiveStatus: mock save, assert status=ACTIVE. (2) restoreOutage_activeOutage_setsRestoredStatusAndTimestamp: mock findById returns ACTIVE outage, after restoreOutage() assert status=RESTORED and actualRestoration is not null. (3) restoreOutage_alreadyRestored_throwsBadRequestException: mock returns RESTORED outage. (4) getActiveOutages_returnsOnlyActiveOnes: mock findByStatus(ACTIVE) returns 2 outages. FieldDispatchServiceTest (3 tests): (1) dispatchTechnician_validOutage_createsDispatch: mock outageRepo returns ACTIVE outage. (2) dispatchTechnician_invalidOutage_throwsResourceNotFoundException: mock returns empty. (3) resolveDispatch_setsResolvedStatusAndTimestamp: assert status=RESOLVED + timestamp set. All 7 tests pass with mvn test. @Slf4j verified in both service classes.

FRONTEND INTERVIEW QUESTIONS — Grid Ops Dashboard + Outage Tracker**Q8. How do contextual status buttons work in FieldDispatchPage?**

A: Each dispatch row renders buttons conditionally based on the current dispatch status. I use a conditional rendering pattern: if status === 'DISPATCHED', render a Mark On-Site button. if status === 'ONSITE', render a Mark Resolved button. if status === 'RESOLVED', render only the status badge with no action button. Clicking a button calls window.confirm with an appropriate message. On confirmation, it calls dispatchService.updateStatus(dispatchId, nextStatus). On success: setDispatches(prev => prev.map(d => d.id === dispatchId ? {...d, status: nextStatus} : d)). The spread operator ensures immutability — React detects the change and re-renders.

Q9. How is the summary row at the top of OutageListPage calculated?

A: The summary row has three computed values derived from the outages array in state: Active Count: outages.filter(o => o.status === 'ACTIVE').length — number of ongoing outages. Total Affected Accounts: outages.reduce((sum, o) => sum + o.affectedAccounts, 0) — total consumers currently without power across all active outages. Average Resolution Time: filter only RESTORED outages, map each to (new Date(o.actualRestoration) - new Date(o.startTime)) in milliseconds, average the values, convert to hours and minutes. All three are pure JavaScript computations on the already-fetched outages array — no additional API calls. They update automatically when state changes.

Q10. How does the Restore button work with confirmation?

A: In OutageListPage, each row with status=ACTIVE has a Restore button (red Bootstrap button). Clicking it calls: if (window.confirm('Restore outage for Zone ' + outage.zoneId + '?')). On confirmation, it calls outageService.restoreOutage(outageId) which sends PATCH /api/outage-events/{id}/restore. On HTTP 200 success: update state to set that outage's status to RESTORED and capture actualRestoration from the response. On HTTP 400 (already restored), display an error alert. The summary row automatically recalculates because the outages state has changed.

SECTION 3 — GENERAL BACKEND INTERVIEW QUESTIONS

Spring Boot Core

Q1. What is Spring Boot and how is it different from Spring Framework?

A: Spring Framework is a comprehensive Java framework that requires significant XML or Java-based configuration for every component — you must manually configure the dispatcher servlet, data sources, transaction managers, etc. Spring Boot adds an opinionated convention-over-configuration layer: auto-configuration (reads the classpath and auto-configures beans), starter dependencies (spring-boot-starter-web pulls Tomcat + Jackson + Spring MVC in one dependency), an embedded server (Tomcat, Jetty, or Undertow bundled inside the JAR), and production-ready features like Actuator. In EnergyGrid, I added spring-boot-starter-web and it automatically configured the embedded Tomcat on port 8095, the DispatcherServlet, and Jackson for JSON serialization.

Q2. What is @SpringBootApplication and what three annotations does it combine?

A: @SpringBootApplication is a convenience meta-annotation combining three: (1) @Configuration: marks the class as a source of @Bean method definitions for the ApplicationContext. (2) @EnableAutoConfiguration: tells Spring Boot to start auto-configuring beans based on classpath contents, other beans, and property settings. (3) @ComponentScan: scans the current package and all sub-packages for classes annotated with @Component, @Service, @Repository, @Controller, and registers them as beans. The class with this annotation is the entry point — SpringApplication.run(App.class, args) creates the ApplicationContext, starts the embedded Tomcat, and makes the application ready.

Q3. What is application.properties and what did you configure in EnergyGrid?

A: application.properties is the central configuration file loaded automatically by Spring Boot from the classpath (src/main/resources). Key EnergyGrid configurations: server.port=8095 (custom port instead of default 8080). spring.datasource.url=jdbc:mysql://localhost:3306/energygrid_monolithic_db. spring.datasource.username=root and spring.datasource.password=root. spring.jpa.hibernate.ddl-auto=update (Hibernate updates schema without dropping data). spring.jpa.show-sql=true (logs all generated SQL for debugging). spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect. jwt.secret=energygrid_secret_key (read with @Value). jwt.expiry.ms=86400000 (24 hours in milliseconds).

Q4. What is @Value and how did you use it for JWT configuration?

A: @Value('\${propertyKey}') injects a value from application.properties into a Spring bean field at application startup. In JwtUtil.java: @Value('\${jwt.secret}') private String secret; and @Value('\${jwt.expiry.ms}') private long expiryMs. Spring reads these values from application.properties and injects them when JwtUtil is instantiated as a singleton bean. This externalises configuration — you can change the JWT secret or expiry time by editing application.properties without modifying or recompiling Java code. In production you would supply these via environment variables or a secrets manager.

Q5. What is the difference between @Entity and a DTO?

A: @Entity is a JPA-managed class mapped to a database table. Hibernate tracks changes to @Entity instances within a transaction and automatically flushes them to the database. Entities may contain sensitive fields (like password in User), Hibernate-specific annotations that pollute the API model, circular references causing infinite JSON serialization, or more fields than the API consumer needs. A DTO (Data Transfer Object) is a plain Java class with no JPA annotations, used purely to carry data between layers. In EnergyGrid: BillRequest DTO receives the POST body, BillingService maps it to a Bill @Entity for persistence, and the controller returns a BillResponse DTO — hiding internal implementation details.

JPA and Hibernate

Q6. Explain spring.jpa.hibernate.ddl-auto and its options.

A: This setting controls how Hibernate manages the database schema on startup. Options: create: drops all tables and recreates from entity definitions every time — loses all data. create-drop: creates on startup, drops on shutdown — useful for in-memory test DBs. update: compares existing schema with entity definitions and applies incremental

changes (adds new columns, creates new tables) without dropping existing data — used in EnergyGrid development. validate: validates that the schema matches entity definitions but makes no changes — throws an error on startup if they do not match. Used in production to detect deployment issues. none: Hibernate does nothing to the schema. For production, use validate or none with a proper migration tool like Flyway or Liquibase.

Q7. What is the difference between EAGER and LAZY loading? When does N+1 occur?

A: EAGER loading fetches the related entity immediately when the parent is loaded — @ManyToOne is EAGER by default. When you load a Bill, Hibernate immediately queries for its associated Tariff in the same or a separate SQL. LAZY loading defers the fetch until the property is first accessed — @OneToMany is LAZY by default. N+1 problem: if you load a list of 50 Bills (1 query), and each bill's tariff is accessed in a loop, Hibernate fires 50 individual SELECT queries = 51 total. Fixes: use JPQL JOIN FETCH (SELECT b FROM Bill b JOIN FETCH b.tariff) to load in one query, use @EntityGraph to specify fetch plan, or use @BatchSize(size=10) to batch related entity fetches. In EnergyGrid, Bills list endpoints use DTOs to avoid triggering lazy loads.

Q8. Explain Spring Data JPA derived query methods with examples from your project.

A: Spring Data JPA parses method names at application startup and generates SQL automatically. Pattern: findBy + FieldName + Condition + And/Or + FieldName. Examples from EnergyGrid: findByEmail(String email) generates SELECT * FROM users WHERE email = ?. findByStatus(BillStatus status) generates SELECT * FROM bills WHERE status = ?. findByAccountIdAndStatus(Integer accountId, BillStatus status) generates SELECT * FROM bills WHERE account_id = ? AND status = ?. findTop1ByMeter_MeterIdOrderByReadingDateDesc(Integer meterId) generates SELECT * FROM meter_readings WHERE meter_id = ? ORDER BY reading_date DESC LIMIT 1. The underscore in Meter_MeterId tells Spring Data to traverse the @ManyToOne meter relationship to access the meterId field.

Q9. What is @Transactional and what are its key attributes?

A: @Transactional wraps a method in a database transaction managed by Spring's transaction proxy. Spring creates an AOP proxy around the @Service bean — the proxy begins a transaction before the method, commits on normal return, and rolls back on unchecked RuntimeException (by default). Key attributes: readOnly=true optimises read-only queries (Hibernate skips dirty checking). propagation: REQUIRED (join existing transaction or create new — the default), REQUIRES_NEW (always create a new transaction, suspending any existing one), NESTED (nested savepoint). isolation: READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE. rollbackFor: specify which exceptions (including checked exceptions) trigger rollback. Used in EnergyGrid in PaymentService and FieldDispatchService for multi-table atomicity.

REST API and Spring MVC

Q10. Explain @RestController, and the difference between @GetMapping, @PostMapping, @PatchMapping.

A: @RestController is @Controller + @ResponseBody — every method automatically serialises its return value to JSON (using Jackson ObjectMapper) and writes it directly to the HTTP response body. There is no view resolution. @GetMapping maps HTTP GET requests — used for data retrieval. @PostMapping maps HTTP POST — used for creating resources, returns 201 Created. @PutMapping maps HTTP PUT — full resource replacement. @PatchMapping maps HTTP PATCH — partial resource update. @DeleteMapping maps HTTP DELETE — resource removal. In EnergyGrid, PATCH is used for status updates (PATCH /bills/{id}/status) because only one field changes, while POST is used for all resource creation endpoints.

Q11. What is the difference between @PathVariable, @RequestParam, and @RequestBody?

A: @PathVariable extracts a value from the URI template path segment. Example: @GetMapping('/api/bills/{id}') with @PathVariable Long id in the method parameter extracts the ID from /api/bills/42. @RequestParam extracts a value from the HTTP query string. Example: @GetMapping('/api/bills') with @RequestParam(required=false) Integer accountId extracts from /api/bills?accountId=5. Optional params use required=false with defaultValue. @RequestBody deserialises the HTTP request body (JSON) into a Java object using Jackson. Example: @PostMapping('/api/bills') with @RequestBody BillRequest request maps the POST body JSON to the BillRequest DTO.

Q12. What HTTP status codes did you use and what is the correct semantic for each?

A: 200 OK: successful GET and successful PATCH operations that return a body. 201 Created: successful POST that creates a new resource — return with the created entity. 204 No Content: successful DELETE that returns no body (e.g., deactivate user). 400 Bad Request: client error — duplicate payment attempt, already-restored outage,

invalid input, failed validation. 401 Unauthorized: missing or invalid JWT token — returned by JwtFilter. 403 Forbidden: authenticated but insufficient role (for future method-level security). 404 Not Found: resource ID does not exist in the database — thrown by ResourceNotFoundException. 500 Internal Server Error: unexpected exception caught by the global exception handler catch-all.

Q13. How does GlobalExceptionHandler work and why is it important?

A: GlobalExceptionHandler is a class annotated @RestControllerAdvice (which combines @ControllerAdvice and @ResponseBody). It contains @ExceptionHandler methods for each custom exception type: handleResourceNotFound(ResourceNotFoundException ex) returns ResponseEntity with HTTP 404 and JSON body {error: 'Not Found', message: ex.getMessage()}. handleBadRequest(BadRequestException ex) returns 400. handleUnauthorized returns 401. A catch-all handleException(Exception ex) returns 500. Without this, Spring would return its default error JSON which leaks internal details. With it, controllers have no try-catch blocks — they simply call services and let exceptions propagate. This follows the separation of concerns principle.

Security, AOP, Logging, and Testing

Q14. What is a JWT? What are the three parts and how do you validate it?

A: JWT (JSON Web Token) is a compact, URL-safe token format. Three Base64URL-encoded parts separated by dots. Header: {alg: HS256, typ: JWT} — specifies signing algorithm. Payload (Claims): {sub: user@email.com, role: ADMIN, iat: issued-at-timestamp, exp: expiry-timestamp}. Signature: HMAC-SHA256(base64(header) + '.' + base64(payload), secret). Validation in JwtUtil: parse the token using the secret key, verify signature integrity, check that exp (expiry) is in the future. If the signature is invalid or the token is expired, JWTs library throws an exception that JwtFilter catches and responds with HTTP 401. The signature ensures the payload has not been tampered with.

Q15. What is CORS and how did you configure it?

A: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks JavaScript from making requests to a different origin (different protocol, domain, or port). The React frontend on localhost:3000 calling the API on localhost:8095 is a cross-origin request — the browser sends a preflight OPTIONS request, and without CORS headers on the server response, the browser blocks the actual request. In EnergyGrid SecurityConfig, I configured a CorsConfigurationSource: allowedOrigins = [http://localhost:3000], allowedMethods = [GET, POST, PUT, PATCH, DELETE, OPTIONS], allowedHeaders = [Authorization, Content-Type], allowCredentials = true. Applied to the security filter chain via http.cors(cors -> cors.configurationSource(source)).

Q16. What is AOP? Explain your LoggingAspect.

A: AOP (Aspect-Oriented Programming) separates cross-cutting concerns from business logic. Instead of adding logging code to every service method (code duplication), I define it once in LoggingAspect annotated @Aspect and @Component. @Before advice: runs before any method matching the pointcut expression execution(*com.cts.energygrid.service.impl.*(..)) — logs the method name and arguments. @AfterReturning advice: runs after successful method return — logs the method name and return value. AuditLogServiceImpl is excluded from the pointcut by adding && !within(AuditLogServiceImpl) to prevent recursive audit log entries when AuditLogService itself is called. This means all 10+ service impl classes automatically get enter/exit logging without any logging code inside them.

Q17. Explain the difference between @Before, @After, @AfterReturning, @AfterThrowing, and @Around.

A: @Before: executes before the matched method — cannot prevent execution unless exception thrown. @After: executes after the method regardless of outcome (like a finally block). @AfterReturning: executes only when the method returns normally — can access the return value via the returning attribute. @AfterThrowing: executes only when the method throws an exception — can access the exception via the throwing attribute. @Around: wraps the method entirely — you call proceedingJoinPoint.proceed() to execute the target method. Gives full before+after control, can modify args and return value, can suppress or replace exceptions. EnergyGrid uses @Before and @AfterReturning for logging.

Q18. How does JUnit testing with Mockito work? Explain with an example from your tests.

A: @ExtendWith(MockitoExtension.class) enables Mockito's JUnit 5 extension — no need to call MockitoAnnotations.openMocks(this) manually. @Mock creates a Mockito mock object that returns default values (null for objects, 0 for numbers, empty list for collections). @InjectMocks creates an instance of the class under test and injects all @Mock fields via constructor, setter, or field injection. Stubbing:

when(billRepository.findById(1L)).thenReturn(Optional.of(mockBill)) makes the mock return a specific value. Throwing: doThrow(new BadRequestException("Paid")).when(paymentService).recordPayment(any()). Verification: verify(billRepository, times(1)).save(any(Bill.class)) asserts the save method was called exactly once. This approach tests service logic in complete isolation.

SECTION 4 — GENERAL FRONTEND INTERVIEW QUESTIONS

React Core Concepts

Q1. What is React and what are its core concepts?

A: React is a JavaScript library for building component-based user interfaces. Core concepts: Components — reusable functions (or classes) that return JSX and can be composed to build complex UIs. JSX — JavaScript XML, an HTML-like syntax compiled by Babel to `React.createElement` calls. Props — read-only data passed from parent to child components, like function arguments. State — mutable data managed within a component using `useState`; changing state triggers re-render. Virtual DOM — React's in-memory representation of the UI; React diffs the virtual DOM before and after state changes and applies only the minimal set of real DOM updates for performance. Unidirectional data flow — data flows down from parent to child via props; events flow up via callback functions.

Q2. What is the difference between `useState` and `useEffect`? Give examples.

A: `useState` manages local component state. `const [bills, setBills] = useState([])` creates a state variable `bills` initialised to empty array and a setter `setBills`. Calling `setBills` with a new value triggers a re-render. `useEffect` manages side effects that run after render. Three forms: `useEffect(() => { fetchBills(); }, [])` runs once on mount (empty array). `useEffect(() => { recalculate(); }, [units, tariff])` runs whenever `units` or `tariff` changes. `useEffect(() => { const id = setInterval(poll, 30000); return () => clearInterval(id); }, [])` runs on mount and the return function (cleanup) runs on unmount. In `EnergyGrid`, `useEffect` with `[]` fetches initial page data, and `onChange` handlers call `useState` setters.

Q3. Explain controlled vs uncontrolled components.

A: Controlled component: the form input's value is driven entirely by React state. The input has `value={name}` so React controls what it displays, and `onChange={e => setName(e.target.value)}` updates state on every keystroke. React is the single source of truth. This enables real-time validation, conditional submit button enabling, dynamic field calculation (like the bill amount in `GenerateBillModal`). Uncontrolled component: the DOM manages its own state; you access values via React refs using `useRef()` and `ref.current.value` only when needed (e.g., on form submit). Controlled is recommended for React forms. All modals in `EnergyGrid` use controlled components.

Q4. What is React Context API and how did you use it for authentication?

A: Context API provides a way to pass data through the component tree without prop drilling (passing through every intermediate component as props). `createContext()` creates the context object. The `Provider` component wraps the tree and supplies a value prop. `useContext()` consumes the value in any descendant component without needing props at each level. In `EnergyGrid`: `AuthContext.jsx` creates the context. `AuthProvider` wraps `App.jsx`, providing `{user, token, role, userId, login, logout}`. Any component — including deeply nested ones like `NotificationBell` inside `Navbar` inside `Layout` — can call `useAuth()` to access the auth state directly without any prop passing through `Layout` and `Navbar`.

Q5. How does React Router DOM work and how did you structure routing?

A: React Router DOM v6 provides client-side routing — URL changes without full page reloads. The browser does not send a new request to the server; React intercepts the URL change and renders the matching component. Structure in `App.jsx`: `BrowserRouter` wraps the entire app. `Routes` contains Route definitions: `Route path='/login'` renders `LoginPage`, `Route path='/bills'` renders `BillListPage` wrapped in `ProtectedRoute`. Nested routes allow shared layouts — all authenticated pages share the `Layout` component (`Navbar` + `Sidebar`) as the parent route. `useNavigate()` programmatically navigates after login success: `navigate('/dashboard')` based on role. `NavLink` with `activeClassName` shows which route is active in the sidebar.

Q6. How did you configure the Axios instance and its interceptor?

A: In `src/services/api.js`: `const api = axios.create({baseURL: 'http://localhost:8095', headers: {'Content-Type': 'application/json'}})`. Request interceptor: `api.interceptors.request.use(config => { const token = localStorage.getItem('token'); if (token) { config.headers.Authorization = 'Bearer ' + token; } return config; }, error => Promise.reject(error))`. Response interceptor: `api.interceptors.response.use(response => response, error => { if (error.response && error.response.status === 401) { logout(); navigate('/login'); } return Promise.reject(error); })`. All service files (`billService.js`, `paymentService.js`) import and use this `api` instance so they automatically get JWT

headers and 401 handling.

Q7. What are React hooks and which ones did you use?

A: Hooks are functions that let functional components use React features previously only available in class components. Rules: only call hooks at the top level of a component (not inside loops, conditions, or nested functions); only call in React functions (not regular JavaScript). Hooks used in EnergyGrid: `useState` — local state management in every component. `useEffect` — data fetching on mount, polling intervals, side effects. `useContext` — consuming `AuthContext` via the custom `useAuth()` hook. `useNavigate` — programmatic navigation from React Router DOM. Custom hook `useAuth()` — wraps `useContext(AuthContext)` and returns auth state and functions, providing a cleaner API than calling `useContext` directly.

Q8. Explain loading and error state patterns used in your pages.

A: Every data-fetching page follows a three-state pattern. State variables: `const [data, setData] = useState([]); const [loading, setLoading] = useState(true); const [error, setError] = useState(null)`. In `useEffect`: `try { setLoading(true); const response = await api.get('/api/bills?accountId=' + accountId); setData(response.data); } catch (err) { setError(err.response?.data?.message || 'Failed to load data'); } finally { setLoading(false); }`. JSX rendering: if loading is true, render a Bootstrap Spinner. If error is not null, render a Bootstrap Alert with the error message. Otherwise render the Table with data. This pattern gives users clear feedback on the API call state.

Q9. What is Bootstrap 5 and how did you integrate it?

A: Bootstrap 5 is a CSS framework providing pre-built responsive components and a 12-column grid system, requiring no jQuery (unlike Bootstrap 4). Integration: `npm install bootstrap`, then import `'bootstrap/dist/css/bootstrap.min.css'` in `src/index.js`. Components used in EnergyGrid: `Container/Row/Col` for responsive layout, `Table` for data tables with striped and bordered variants, `Badge` for color-coded status (`bg-success` for PAID, `bg-danger` for OVERDUE, `bg-warning` for DISPUTED), `Modal` for forms (`GenerateBillModal`, `RecordPaymentModal`), `Card` for KPI panels in the dashboard, `Navbar` for navigation, `NavTabs` for category filter tabs, `Button` variants for primary/secondary/danger actions. Bootstrap's mobile-first responsive grid automatically adapts layouts to different screen sizes.

SECTION 5 — COMPLETE SPRING BOOT QUESTIONS (All Topics Interviewers Ask)

Core IoC, Beans and Dependency Injection

Q1. What is Inversion of Control (IoC)?

A: IoC is a design principle where the control of object creation and lifecycle management is inverted — instead of objects creating their own dependencies, a container (Spring's `ApplicationContext`) creates and injects them. This leads to loosely coupled code. In `EnergyGrid`: `BillingService` does not create a `BillRepository` instance with new keyword; Spring creates it and injects it via `@Autowired`. The service just declares what it needs.

Q2. What is Dependency Injection and what are the three types in Spring?

A: DI is the mechanism implementing IoC — the container supplies a bean's dependencies. Types: (1) Constructor Injection — dependencies in the constructor parameters. Best for mandatory dependencies; enables immutability (final fields); makes dependencies explicit. (2) Setter Injection — `@Autowired` on a setter method. Good for optional dependencies. (3) Field Injection — `@Autowired` directly on the field. Most concise but hides dependencies, makes unit testing slightly harder (no constructor to inject mocks), and is not recommended by Spring team for production code. `EnergyGrid` uses field injection for brevity.

Q3. What is @Component and what are its specialisations?

A: `@Component` is the base stereotype annotation that registers a class as a Spring bean for component scanning. Specialisations: `@Service` marks the business logic layer — semantically meaningful, no extra functionality. `@Repository` marks the data access layer and adds persistence exception translation (converts JPA/Hibernate exceptions to Spring's `DataAccessException` hierarchy). `@Controller` marks the web layer (Spring MVC controller). `@RestController` = `@Controller` + `@ResponseBody` for REST APIs. Using the correct annotation improves code readability and enables any future framework enhancements specific to that layer.

Q4. What is Bean Scope? Explain Singleton and why it is safe.

A: Bean scope defines the lifecycle and number of instances. Singleton (default): Spring creates exactly one instance of the bean per `ApplicationContext` and shares it across all requests. Prototype: a new instance is created every time the bean is requested. Web scopes: Request (one per HTTP request), Session (one per HTTP session), Application (one per `ServletContext`). Singleton is safe in `EnergyGrid` because all `@Service` and `@Repository` beans are stateless — they do not store request-specific data in instance fields. Never store request data in a Singleton field — it causes race conditions and data leaks.

Q5. What is @Autowired and when do you need @Qualifier?

A: `@Autowired` tells Spring to inject a matching bean by type. If exactly one bean of the required type exists in the context, Spring injects it. If multiple beans of the same type exist (for example, two `@Repository` implementations of the same interface), `@Qualifier("specificBeanName")` specifies which one to inject. `@Primary` on a bean marks it as the default candidate when no `@Qualifier` is specified. From Spring 4.3+, if a class has a single constructor, `@Autowired` is optional — Spring automatically uses constructor injection.

Q6. What is @Bean and when do you use it instead of @Component?

A: `@Bean` is a method-level annotation placed inside a `@Configuration` class. It registers the method's return value as a Spring bean. Use `@Bean` when: you cannot annotate the class with `@Component` (it is a third-party class like `BCryptPasswordEncoder` or `RestTemplate`). You need programmatic configuration — conditionally creating different bean implementations based on environment. Example in `EnergyGrid` `SecurityConfig`: `@Bean` `public BCryptPasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }` registers the encoder as a singleton bean injectable anywhere in the application.

Q7. What is Spring Boot Auto-Configuration?

A: `@EnableAutoConfiguration` reads auto-configuration candidate classes from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Spring Boot 3). Each listed class is a `@Configuration` annotated with `@Conditional` annotations: `@ConditionalOnClass` (activate if class X is on classpath), `@ConditionalOnMissingBean` (activate if no bean of type Y is already defined), `@ConditionalOnProperty`

(activate if property Z has a specific value). For example, `DataSourceAutoConfiguration` activates only if a JDBC driver is on the classpath and no `DataSource` bean is already defined. Run with `--debug` flag to see all positive and negative auto-configuration matches in the startup log.

Q8. What are Spring Boot Starters?

A: Starters are curated sets of dependency descriptors that pull in all needed libraries with compatible tested versions in one `pom.xml` entry. `spring-boot-starter-web`: `spring-webmvc`, embedded Tomcat, Jackson JSON, Hibernate Validator. `spring-boot-starter-data-jpa`: Spring Data JPA, Hibernate ORM, JPA API. `spring-boot-starter-security`: Spring Security core. `spring-boot-starter-test`: JUnit 5, Mockito, AssertJ, Spring Test. `EnergyGrid` `pom.xml` uses 4 starters plus `jjwt` for JWT and Lombok. Without starters you would need to manage 15+ individual dependencies and their version compatibility manually.

Q9. What is Lombok and which annotations do you use?

A: Lombok is an annotation processor that generates boilerplate Java code at compile time. Annotations used in `EnergyGrid`: `@Data` on entities — generates getters, setters, equals, hashCode, and toString. `@Builder` on entities — generates the Builder pattern (`User.builder().name('x').email('y').role(ADMIN).build()`) for readable object creation. `@NoArgsConstructor` — generates default constructor (required by JPA). `@AllArgsConstructor` — generates constructor with all fields (required by `@Builder`). `@Slf4j` on all `@Service` classes — generates a static final Logger field named `log`. Lombok annotations are processed at compile time — zero runtime overhead.

Q10. What is @Slf4j and how does logging work?

A: `@Slf4j` is a Lombok annotation that generates: `private static final Logger log = LoggerFactory.getLogger(ClassName.class)`. In service methods: `log.info('Generating bill for accountId: {}', accountId)` logs at INFO level with parameterised message (lazy evaluation — the string is not built if INFO is disabled). `log.error('Error processing payment: {}', e.getMessage(), e)` logs exception with stack trace. Log levels (lowest to highest): TRACE, DEBUG, INFO, WARN, ERROR. Spring Boot's default level is INFO. Logback is the default implementation. Configure in `application.properties`: `logging.level.com.cts.energygrid=DEBUG`.

JPA Deep Dive

Q11. What is Spring Data JPA and what does JpaRepository give you?

A: Spring Data JPA is a Spring module that eliminates boilerplate DAO (Data Access Object) code. By declaring an interface that extends `JpaRepository[T, ID]`, Spring generates a runtime proxy implementation at startup. Built-in methods: `save(entity)`, `findById(id)` returning `Optional`, `findAll()` and `findAll(Pageable)` and `findAll(Sort)`, `deleteById(id)`, `count()`, `existsById(id)`, `saveAll(Iterable)`, `deleteAll()`. Additionally: derived query methods from method names, `@Query` for custom JPQL or native SQL, `@Modifying` for update/delete queries. In `EnergyGrid`, every entity has its own repository interface with zero implementation code.

Q12. What is @CreationTimestamp vs @UpdateTimestamp?

A: `@CreationTimestamp` is a Hibernate annotation that automatically sets the annotated field to the current date/time when the entity is first INSERT-ed into the database. The application code does not need to set this value — Hibernate handles it. Used in `EnergyGrid`: `AuditLog.timestamp`, `Notification.createdDate`, `FieldDispatch.dispatchTime`. `@UpdateTimestamp` automatically updates the field to current time on every UPDATE operation. Both work with `LocalDateTime`, `Date`, `Calendar`, or `Timestamp` Java types. They eliminate `LocalDateTime.now()` calls in service code.

Q13. What is the difference between save() and saveAndFlush()?

A: Both persist an entity, but they differ in timing. `save()` marks the entity as managed by Hibernate and schedules the SQL INSERT or UPDATE to be flushed to the database at the end of the transaction or at the next flush point. Hibernate may batch multiple saves. `saveAndFlush()` immediately executes the SQL statement within the current transaction without waiting for the transaction commit. Use `saveAndFlush` when: you need the auto-generated primary key immediately after save for use in the same transaction, you want to detect database constraint violations immediately, or subsequent logic in the same transaction depends on the saved data being visible in the database.

Q14. How does JPA handle enum persistence and why must you use EnumType.STRING?

A: Without `@Enumerated` annotation, JPA defaults to `ORDINAL` which stores the enum's position index (0, 1, 2...). `@Enumerated(EnumType.ORDINAL)` is dangerous: if you insert a new enum constant between existing ones, all

subsequent ordinals shift and every existing database record maps to the wrong constant — a silent data corruption bug. `@Enumerated(EnumType.STRING)` stores the constant name ('ACTIVE', 'CONSUMER') as VARCHAR. Adding new constants or reordering never affects existing data. All 20 enums in EnergyGrid use `EnumType.STRING`. The VARCHAR column in MySQL must be sized to hold the longest name.

Q15. What is JPQL and how does it differ from SQL?

A: JPQL (Java Persistence Query Language) is an object-oriented query language that operates on entity classes and their fields, not on database tables and columns. Example: JPQL: `SELECT b FROM Bill b WHERE b.accountId = :accountId AND b.status = :status`. SQL: `SELECT * FROM bills WHERE account_id = ? AND status = ?`. JPQL uses the Java class name `Bill` (not table `bills`), and field name `accountId` (not `account_id`). Hibernate translates JPQL to the correct SQL dialect at runtime — the same JPQL works on MySQL, PostgreSQL, H2, and Oracle without modification. Use `@Query(value='...', nativeQuery=true)` for database-specific SQL when JPQL is insufficient.

Q16. How does pagination work in Spring Data JPA?

A: Spring Data supports pagination through the `Pageable` interface. Repository method: `Page[AuditLog] findAll(Pageable pageable)` — inherited from `JpaRepository` or declared with filters. Controller creates `PageRequest`: `Pageable pageable = PageRequest.of(page, size, Sort.by("timestamp").descending())`. Pass to repository. The returned `Page[T]` contains: `getContent()` — the list of items for the current page, `getTotalElements()` — total records, `getTotalPages()` — total pages, `getNumber()` — current page number (0-indexed), `getSize()` — page size. Controller returns this `Page` directly or maps it to a response DTO. Used in EnergyGrid for `GET /api/audit-logs` with page and size query parameters.

Spring Security Deep Dive

Q17. How does Spring Security's filter chain work?

A: Spring Security wraps the servlet container with a `DelegatingFilterProxy` that delegates to a `SecurityFilterChain`. The chain contains multiple filters executed in order before any Spring MVC controller runs. Key filters in EnergyGrid's chain: `CorsFilter` — handles CORS preflight OPTIONS requests. `JwtFilter` (custom, before `UsernamePasswordAuthenticationFilter`) — validates JWT and sets authentication in `SecurityContextHolder`. `ExceptionTranslationFilter` — catches `AccessDeniedException` and `AuthenticationException`. `AuthorizationFilter` — checks if the authenticated user has permission for the requested URL. `SecurityContextHolder` uses `ThreadLocal` storage — each thread has its own authentication context, ensuring thread safety under concurrent requests.

Q18. What is `OncePerRequestFilter` and why does `JwtFilter` extend it?

A: `OncePerRequestFilter` is a Spring base class that guarantees the `doFilterInternal()` method executes exactly once per HTTP request, regardless of whether the request goes through servlet forwards or includes (which can trigger filters multiple times in some containers). `JwtFilter` extends `OncePerRequestFilter` and overrides `doFilterInternal()`. Inside it: extracts the Authorization header, checks for 'Bearer ' prefix, extracts the token, validates it with `JwtUtil`, extracts email and role, creates a `UsernamePasswordAuthenticationToken` with the role as a `GrantedAuthority`, sets it in `SecurityContextHolder.getContext().setAuthentication()`. For `/api/auth/**` paths, the filter returns early without validation.

Q19. What is CSRF and why did you disable it in EnergyGrid?

A: CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks an authenticated user's browser into making an unwanted request to your server using the user's existing session cookies. Spring Security protects against this by generating a CSRF token stored in the session and requiring it in state-changing requests. In EnergyGrid, CSRF is disabled (`csrf.disable()`) because: the application is STATELESS — it uses JWT tokens, not session cookies. JWT tokens must be explicitly set in the Authorization header by JavaScript code — they cannot be automatically included by a browser like cookies can. Stateless JWT architecture is inherently protected against CSRF.

Q20. What is method-level security? How would you add it to EnergyGrid?

A: Method-level security adds fine-grained authorisation at the service or controller method level. Enable with `@EnableMethodSecurity` on a configuration class. Annotations: `@PreAuthorize("hasRole('ADMIN')")` on a method — runs before the method and blocks if the condition fails. `@PreAuthorize("hasAnyRole('BILLING','ADMIN')")` for multiple roles. `@PostAuthorize` — checks after method execution. `@Secured({ROLE_ADMIN})`. In `JwtFilter`, roles are set as: `new SimpleGrantedAuthority("ROLE_" + role)`. EnergyGrid currently relies on URL-level security —

method-level would be the next step to prevent a BILLING user from calling delete-user endpoints.

Exception Handling, AOP, Profiles and Advanced Topics

Q21. What is @ControllerAdvice vs @RestControllerAdvice?

A: @ControllerAdvice makes a class a global exception handler and model attribute provider for all @Controller classes. @RestControllerAdvice = @ControllerAdvice + @ResponseBody — exception handler methods automatically serialise their return values to JSON. EnergyGrid uses @RestControllerAdvice on GlobalExceptionHandler. Each method annotated with @ExceptionHandler(SomeException.class) handles that exception type from any controller: @ExceptionHandler(ResourceNotFoundException.class) returns ResponseEntity with status 404 and JSON body. Spring matches the most specific exception type first.

Q22. What is circular dependency in Spring and how do you fix it?

A: Circular dependency occurs when bean A requires bean B and bean B requires bean A — Spring cannot instantiate either because each requires the other to exist first. Spring Boot 2.6+ fails fast on startup with a circular dependency error. Fixes: (1) Refactor — extract the shared logic into a third service C that neither A nor B depends on. This is the cleanest solution. (2) @Lazy on one of the @Autowired fields — Spring injects a proxy initially and resolves the real bean on first use. (3) Setter injection instead of constructor injection — Spring can create A without B, then set B afterwards. In EnergyGrid, AuditLogService is called by many services but never calls back into them — no circular dependency.

Q23. What is Spring Profiles and how would you use them in EnergyGrid?

A: Profiles allow different configurations for different environments. Activate with spring.profiles.active=dev in application.properties or as a JVM argument -Dspring.profiles.active=prod. @Profile('dev') on a @Configuration class or @Bean activates it only for the dev profile. application-dev.properties overrides application.properties when dev profile is active. In EnergyGrid: dev profile — MySQL at localhost, show-sql=true, logging level DEBUG. test profile — H2 in-memory database (spring-boot-starter-h2), ddl-auto=create-drop — no real MySQL needed for JUnit tests. prod profile — MySQL at production server, show-sql=false, logging level WARN, Actuator endpoints secured.

Q24. What is Spring Boot Actuator?

A: Actuator adds production-ready monitoring and management endpoints to Spring Boot. Dependency: spring-boot-starter-actuator. Built-in endpoints: /actuator/health — application health with database and diskSpace indicators. /actuator/info — custom app info from application.properties. /actuator/metrics — JVM memory, HTTP request counts, Hikari pool stats. /actuator/beans — all Spring beans in the context. /actuator/env — resolved property values. /actuator/mappings — all URL handler mappings (useful for debugging). Enable all: management.endpoints.web.exposure.include=*. In production, secure Actuator behind /management path with HTTP Basic auth.

Q25. What is Bean Validation (@Valid) and how do you use it?

A: Bean Validation (JSR-380) validates request body fields using constraint annotations on DTO classes. Add spring-boot-starter-validation dependency. On DTO: @NotNull private Integer accountId; @NotBlank @Size(max=150) private String name; @Email private String email; @Positive private BigDecimal amount. In controller: @PostMapping with @Valid @RequestBody BillRequest request. On validation failure, Spring throws MethodArgumentNotValidException. Handle in @RestControllerAdvice: @ExceptionHandler(MethodArgumentNotValidException.class) extracts field errors and returns 400 with JSON {field: 'amount', message: 'must be positive'}.

Q26. What happens if you use @Transactional on a private method?

A: @Transactional works through Spring AOP proxying. When you call a @Transactional method on a Spring bean from outside the bean, Spring's proxy intercepts the call and wraps it in a transaction. However: (1) If you call a @Transactional method from within the same class (self-invocation), the call bypasses the proxy and the transaction annotation is ignored. (2) @Transactional on private methods is silently ignored because Spring's CGLIB proxy cannot override private methods. To fix self-invocation: inject the bean into itself (using @Lazy to avoid circular dependency) or use ApplicationContext to get the proxy. Best practice: @Transactional on public methods in @Service classes only.

Q27. What is the complete lifecycle of a Spring Boot request? Explain all layers.

A: HTTP request arrives at embedded Tomcat. Tomcat passes to DelegatingFilterProxy (Spring Security filter chain). JwtFilter validates token, sets SecurityContext. Request reaches DispatcherServlet (Spring MVC front controller). DispatcherServlet consults HandlerMapping to find the matching @RestController method by URL and HTTP method. HandlerAdapter invokes the controller method. @RequestBody triggers Jackson ObjectMapper to deserialise JSON body to DTO. Controller calls @Service method. Service calls @Repository method. Repository generates SQL via Hibernate, executes against MySQL. Result maps to entity. Service maps entity to response DTO. Controller returns ResponseEntity. HandlerAdapter invokes HttpMessageConverter (Jackson) to serialise DTO to JSON. Response written to HTTP output stream. Tomcat sends response to client.

Q28. What is @Modifying and when is it needed?

A: @Modifying is required alongside @Query when the JPQL or SQL query is an UPDATE or DELETE statement (not a SELECT). Without @Modifying, Spring Data throws an InvalidDataAccessApiUsageException because it expects SELECT queries to return results. Example: @Modifying @Transactional @Query('UPDATE Bill b SET b.status = :status WHERE b.dueDate < :today AND b.status = GENERATED') int markOverdueBills(@Param('today') LocalDate today, @Param('status') BillStatus status). @Transactional is also required because update/delete queries must run within a transaction. The return type int gives the count of affected rows. In EnergyGrid, the markOverdue logic is implemented in Java for clarity but @Modifying would be more efficient for large datasets.

Q29. What is CommandLineRunner and when would you use it in EnergyGrid?

A: CommandLineRunner is a functional interface with a single run(String... args) method called by SpringApplication after the ApplicationContext is fully initialised and started. Used for: initial data seeding, startup validation, warming up caches. In EnergyGrid a DataSeeder class implements CommandLineRunner: @Component @Order(1) class DataSeeder implements CommandLineRunner { @Override public void run(String... args) { if (userRepository.count() == 0) { seed 5 users (one per role); seed 2 tariffs; seed test data } } }. @Order(1) ensures this runs before any other CommandLineRunner beans. ApplicationRunner is a similar interface that receives an ApplicationArguments object instead of raw String args — useful for parsing named arguments.

Q30. What is the difference between @SpringBootTest, @WebMvcTest, and @DataJpaTest?

A: @SpringBootTest loads the complete ApplicationContext with all beans, database, security, and web layer. Use for end-to-end integration tests. webEnvironment=RANDOM_PORT starts a real embedded server — use TestRestTemplate for HTTP calls. Slowest. @WebMvcTest loads only the web layer (@Controller, @ControllerAdvice, security config) without @Service or @Repository. Services must be provided via @MockBean. Use MockMvc to test HTTP request/response. Fast for controller layer tests. @DataJpaTest loads only the JPA layer — @Repository beans, Hibernate, and an in-memory H2 database. No @Service or @Controller. Use for testing repository queries in isolation. Each annotation is scoped for speed — use the narrowest one that covers your test scenario.

Microservices, Docker and Cloud (Week 8-9 Syllabus Topics)

Q31. What is the difference between a Monolith and Microservices architecture?

A: Monolith: all modules (IAM, Billing, Payments, etc.) in a single codebase, single deployment unit, single database. Simple to develop and debug initially. Problems at scale: any change requires full redeployment, a bug in one module can crash the entire application, cannot scale individual modules independently. EnergyGrid is currently monolithic. Microservices: each bounded domain (IAM Service, Billing Service, Payment Service) is an independent deployable application with its own database and tech stack. Benefits: independent scaling, fault isolation, team autonomy. Challenges: network latency, distributed transactions (no @Transactional across services), service discovery, distributed tracing.

Q32. What is an API Gateway in microservices?

A: API Gateway is the single entry point for all client requests. Instead of clients knowing the address of every microservice, they send all requests to the gateway. The gateway handles: Routing — /api/bills/* forwarded to Billing Service, /api/auth/* to IAM Service. Authentication — validates JWT before forwarding (centralised auth, services trust the gateway). Rate limiting — throttle requests per client. SSL termination — HTTPS to HTTP internally. Request/response transformation — add correlation IDs, log requests. Popular options: Spring Cloud Gateway (reactive, Spring ecosystem), Kong, AWS API Gateway, Netflix Zuul (older). Without a gateway in EnergyGrid

microservices, the React frontend would need to know 5 different service URLs.

Q33. What is Service Discovery and how does Eureka work?

A: In microservices, service instances start and stop dynamically with changing IP addresses. Service Discovery solves the problem of finding service instances at runtime. Netflix Eureka (Spring Cloud Netflix): Eureka Server is a registry application annotated `@EnableEurekaServer`. Each microservice (Eureka Client) annotated `@EnableDiscoveryClient` registers itself on startup with its name and IP, and sends heartbeats every 30 seconds. When Billing Service needs to call Payment Service, it asks Eureka for all instances of 'payment-service' and gets the current IP and port. Spring Cloud LoadBalancer (successor to Ribbon) distributes requests across multiple instances for load balancing.

Q34. What is Docker and how would you containerise EnergyGrid?

A: Docker packages an application with all its runtime dependencies (JRE, configs) into a portable image that runs consistently in any environment. Dockerfile for EnergyGrid backend: FROM openjdk:17-jre-slim. WORKDIR /app. COPY target/energygrid-0.0.1-SNAPSHOT.jar app.jar. EXPOSE 8095. ENTRYPOINT java -jar app.jar. Build: docker build -t energygrid-backend:1.0 . Run: docker run -p 8095:8095 -e spring.datasource.url=jdbc:mysql://mysql:3306/energygrid_db energygrid-backend:1.0. Docker Compose (docker-compose.yml) orchestrates: a MySQL 8 container and the EnergyGrid app container on a shared network. docker-compose up starts both. This eliminates 'works on my machine' problems.

SECTION 6 — QUICK REFERENCE CHEAT SHEETS

Key Annotations — Complete Reference

Annotation	Layer	Purpose / Notes
@SpringBootApplication	Main Class	@Configuration + @EnableAutoConfiguration + @ComponentScan combined
@Entity	Entity	Maps Java class to DB table (JPA managed)
@Table(name='...')	Entity	Specifies exact DB table name
@Id + @GeneratedValue(IDENTITY)	Entity	Primary key + auto-increment
@Column(name=,nullable=)	Entity	Column mapping with constraints
@ManyToOne + @JoinColumn(name=)	Entity	FK relationship — many-to-one
@OneToMany(mappedBy=)	Entity	Inverse side of one-to-many relationship
@Enumerated(EnumType.STRING)	Entity	Store enum name as VARCHAR — never use ORDINAL
@CreationTimestamp	Entity	Auto-set to now() on INSERT (Hibernate)
@UpdateTimestamp	Entity	Auto-update to now() on every UPDATE
@Data	Entity/DTO	Lombok: getters + setters + equals + hashCode + toString
@Builder	Entity/DTO	Lombok: fluent builder pattern
@NoArgsConstructor	Entity	Required by JPA for entity instantiation
@AllArgsConstructor	Entity	Required alongside @Builder
@Slf4j	Service	Lombok: generates static Logger field named 'log'
@Service	Service	Business logic bean — semantic stereotype
@Transactional	Service	DB transaction — rolls back on RuntimeException
@Transactional(readOnly=true)	Service	Optimise read-only queries
@Repository	Repository	DAO bean + JPA exception translation
@Query('JPQL...')	Repository	Custom JPQL query
@Query(nativeQuery=true)	Repository	Native SQL query
@Modifying + @Transactional	Repository	Required for @Query UPDATE/DELETE
@RestController	Controller	@Controller + @ResponseBody — returns JSON
@RequestMapping('/api/...')	Controller	Base URL path for all methods in class
@GetMapping / @PostMapping	Controller	HTTP GET and POST endpoints
@PutMapping / @PatchMapping	Controller	HTTP PUT (full replace) / PATCH (partial update)
@DeleteMapping	Controller	HTTP DELETE endpoint
@PathVariable	Controller	Extract value from URI path /{id}
@RequestParam(required=false)	Controller	Extract query string parameter ?key=value
@RequestBody	Controller	Deserialise JSON request body to object
@Valid	Controller	Trigger Bean Validation on @RequestBody
@Autowired	Any	Inject Spring bean by type
@Value('\${key}')	Any	Inject property from application.properties

@Qualifier('name')	Any	Specify which bean to inject when multiple exist
@Configuration	Config	Marks class as bean definition source
@EnableWebSecurity	Config	Enables Spring Security configuration
@Bean	Config	Registers method return value as Spring bean
@Aspect + @Component	AOP	Declare AOP aspect class
@Before(pointcut)	AOP	Advice running before matched methods
@AfterReturning(pointcut)	AOP	Advice running after successful return
@Around(pointcut)	AOP	Wrap method — call proceed() to execute target
@RestControllerAdvice	Exception	Global exception handler for all @RestController
@ExceptionHandler(Ex.class)	Exception	Handle specific exception type globally
@ExtendWith(MockitoExtension)	Test	Enable Mockito in JUnit 5
@Mock	Test	Create Mockito mock — no Spring context
@InjectMocks	Test	Create class under test with @Mock fields injected
@MockBean	Test	Spring Boot test mock — replaces bean in context
@SpringBootTest	Test	Load full ApplicationContext for integration test
@WebMvcTest	Test	Load web layer only — mock services with @MockBean
@DataJpaTest	Test	Load JPA layer only — H2 in-memory database
@Profile('dev')	Config	Activate bean/config only for specified profile
@Order(1)	Config	Control bean initialisation order

HTTP Status Codes — Quick Reference

Code	Name	When to use
200	OK	Successful GET, PATCH, PUT responses that return a body
201	Created	Successful POST — new resource created (bill, payment, outage)
204	No Content	Successful DELETE — operation completed, no response body
400	Bad Request	Client error: duplicate payment, already-restored outage, failed validation
401	Unauthorized	Missing or invalid JWT token — JwtFilter returns this
403	Forbidden	Valid token but role not allowed for this endpoint
404	Not Found	Requested resource ID does not exist — ResourceNotFoundException
500	Internal Server Error	Unexpected server-side exception — GlobalExceptionHandler catch-all

JUnit 5 + Mockito Quick Reference

Method / Annotation	Purpose
@ExtendWith(MockitoExtension.class)	Enable Mockito in JUnit 5 test class (replaces MockitoAnnotations.openMocks)
@Mock	Create a Mockito mock that returns default values (null, 0, empty list)
@InjectMocks	Create the class under test with all @Mock fields auto-injected
@Spy	Partial mock — real method implementations unless specifically stubbed
when(mock.method(arg)).thenReturn(val)	Stub the mock to return a value on call
when(mock.method()).thenThrow(new Ex())	Stub the mock to throw an exception
doThrow(ex).when(mock).method(arg)	Stub void methods to throw exceptions

verify(mock, times(1)).method(arg)	Assert the mock method was called exactly N times
verify(mock, never()).method(arg)	Assert the mock method was never called
assertThrows(Ex.class, () -> code)	Assert that executing code throws the specified exception
assertEquals(expected, actual)	Assert two values are equal
assertNotNull(value)	Assert value is not null
assertTrue(condition)	Assert condition is true
ArgumentCaptor.forClass(Type.class)	Capture argument passed to mock for detailed assertion
@BeforeEach	Runs before each test method — use for test setup
@AfterEach	Runs after each test method — use for cleanup

Complete End-to-End Flow — Full Demo Scenario

Step	Actor	Action	API Call	DB Change
1	Admin	Create consumer user	POST /api/users	users table — CONSUMER role
2	Any	Login	POST /api/auth/login	— (JWT returned)
3	Admin	Register consumer account	POST /api/consumer-accounts	consumer_accounts — ACTIVE
4	Consumer	Submit connection request	POST /api/connection-requests	connection_requests — PENDING
5	Admin	Assign field tech	PATCH /connection-requests/{id}/assign?techId=	connection_requests — SCHEDULED
6	Field Tech	Install meter	POST /api/meters	meters — ACTIVE
7	Field Tech	Complete request	PATCH /connection-requests/{id}/status=COMPLETED	connection_requests — COMPLETED
8	Field Tech	Record meter reading	POST /api/meter-readings	meter_readings — VALIDATED
9	Billing	Create RESIDENTIAL tariff	POST /api/tariffs	tariffs — ACTIVE
10	Billing	Generate bill (calc amount)	POST /api/bills	bills — GENERATED
11	Consumer	Make payment	POST /api/payments	payments — RECEIVED; bills — PAID
12	Grid Ops	Report outage in zone	POST /api/outage-events	outage_events — ACTIVE
13	Grid Ops	Dispatch technician	POST /api/field-dispatches	field_dispatches — DISPATCHED
14	Field Tech	Arrive on-site	PATCH /field-dispatches/{id}/status=ONSITE	field_dispatches — ONSITE
15	Field Tech	Resolve issue	PATCH /field-dispatches/{id}/resolve	field_dispatches — RESOLVED
16	Grid Ops	Restore outage	PATCH /outage-events/{id}/restore	outage_events — RESTORED
17	Grid Ops	Check SLA	GET /outage-events/{id}/sla	— (slaMet: true/false)

1 8	Admin	Generate report	POST /api/energy-reports?scope=GRID	energy_reports — metrics JSON
--------	-------	-----------------	-------------------------------------	-------------------------------

INTERVIEW SUCCESS TIPS: (1) Always tie answers back to EnergyGrid — say 'In my project I used X because Y'. (2) For each module, explain in order: database schema FIRST, then entity/JPA, then service logic, then REST endpoints. (3) Know your end-to-end flow cold: consumer registers to account to meter to reading to bill to payment. (4) Be ready to name every JUnit test method you wrote and what it verifies. (5) Always mention `@Slf4j` logging in services and `@Transactional` for multi-table operations — interviewers love these. (6) DTO vs Entity is asked in almost every interview — entity is DB-mapped JPA managed, DTO is clean API contract. (7) Know `@Enumerated(EnumType.STRING)` and WHY STRING not ORDINAL. (8) For frontend: explain controlled components, `useEffect` cleanup, and why Context avoids prop drilling.